5
Amazon Data Analytics Services

Amazon provides analytics tools and services for many forms of data. Continuing from the **Amazon Web Services** (**AWS**) database discussions in the previous chapter, we will focus on the AWS big data analytics services in this chapter.

What is big data? In a nutshell, **big data** refers to big and complex datasets that are difficult to process using traditional data analytics tools. Big data is typically characterized by its volume, velocity, and variety:

- **Volume**: Big data refers to datasets that are too large to be processed using traditional database management systems. The size of big data can range from terabytes to petabytes, and it is often generated in real time.
- **Velocity**: Big data is often generated at a high velocity, meaning that it is created and collected rapidly. This requires real-time or near-real-time processing and analysis to turn the data into meaningful insights.
- **Variety**: Big data comes in many different forms, including structured data, semi-structured data, and unstructured data. It can also include audio and video files, images, and sensor data.

Big data can be collected from a wide range of sources, including social media, online transactions, mobile devices, and sensors. It can provide valuable business insights and opportunities. However, it requires advanced tools and techniques to manage and process, including cloud-based storage, computing and other modern tools. AWS offers a range of data analytics services that can be used to manage big data. In this chapter, we will cover the following AWS big data analytics services:

- AWS Glue and data crawlers
- Amazon Athena
- The AWS Kinesis service family

- Amazon QuickSight
- Amazon **Elastic MapReduce** (**EMR**)

During our discussions in this chapter, we will use sample code to process big data. The examples shown in this chapter are also available in the GitHub repository for this book: `https://github.com/PacktPublishing/Self-Taught-Cloud-computing-Engineer`.

Let us start with the AWS big data pipeline and understand the data life cycle in the cloud, including the ingestion, storing, processing, and visualization steps.

Understanding the AWS big data pipeline

With the rise of big data and the increasing availability of data analytics tools and technologies, data analytics has become an essential component of modern business operations. Cloud-based technologies and services have been widely used to analyze and derive insights from big data, and provide the following benefits:

- **Scalability**: Cloud-based data analytics systems can scale up or down based on the input volume of data and traffic, allowing businesses to handle large-scale datasets without having to invest in expensive hardware and infrastructure
- **Cost-effectiveness**: Cloud-based data analytics systems are typically pay-as-you-go, allowing businesses to only pay for the resources they need and avoid extra investment in expensive hardware and infrastructure
- **Flexibility**: Cloud-based data analytics provides a flexible and agile environment for processing and analyzing data, allowing businesses to select the best from different techniques and algorithms
- **Accessibility**: Cloud-based data analytics can be accessed from anywhere with an internet connection, providing a globally accessible and collaborative environment for analyzing data
- **Security**: Cloud-based data analytics provides strong security features, such as encryption, access controls, and compliance with **General Data Protection Regulation** (**GDPR**) and other requirements

AWS provides a suite of data analytics services to manage big data life cycles:

- **Data ingestion**: In *Chapter 1*, we learned about Amazon Snowball and Snowmobile, which transfer large amounts of data from on-premises to the AWS cloud. In this chapter, we will discuss the following data ingestion services:
    - **AWS Glue**: A fully managed ETL service that can automate the process of discovering and cataloging metadata about data sources, generate ETL code to transform and load data, and schedule and run ETL jobs. As part of the AWS Glue service, an **AWS crawler** detects the schema and structure of data and enables users to automatically discover, categorize, and organize their data in various data stores.

    - **Amazon Kinesis**: A fully managed service that enables real-time streaming data ingestion at scale. With Kinesis, organizations can easily collect, process, and analyze data from various sources, such as website clickstreams, IoT devices, and social media feeds.

- **Data storage**: In previous chapters, we introduced Amazon cloud storage services such as EBS, EFS, S3, and Glacier. We also discussed Amazon database services such as RDS, DynamoDB, ElastiCache, and Redshift.
- **Data processing**: We will cover the following AWS data processing services:
    - **Amazon EMR**: A managed Hadoop framework that allows you to process large amounts of data using distributed computing. It can also be used to run other distributed data processing frameworks, such as Apache Spark and Apache Hive.
    - **Amazon Athena**: A flexible, serverless, interactive service to analyze structured, semi-structured, or structured data stored in Amazon S3, using SQL or Python.
- **Data visualizing**: We will focus on **Amazon QuickSight**, a **business intelligence** (**BI**) service that allows users to create interactive visualizations, dashboards, and reports using a web-based interface. QuickSight supports a wide range of data sources, including AWS data services such as Amazon S3, Amazon Redshift, Amazon Athena, and Amazon RDS.

From data ingestion, storing, and processing to visualization, these Amazon cloud tools provide efficient big data management services. Let's start by looking at an Amazon data ingestion service – Amazon Glue and data crawlers.

AWS Glue

As we explained earlier, AWS Glue is an ETL process used to extract data from various sources, transform it into a consistent format and structure, and then load it into a target data repository, such as an S3 bucket or a data warehouse. In an ETL process such as the one used in AWS Glue, the data is typically transformed before it is loaded into the target database. AWS Glue has the following features:

- Automatically generate schemas from semi-structured data by using crawlers, which run on your data sources, derive a schema from them, and populate the Data Catalog. Crawlers can run on many data stores, including Amazon S3, Amazon Redshift, most relational databases, and DynamoDB. By using the metadata in the Data Catalog, you can also automatically generate scripts with AWS Glue extensions as the starting point of your AWS Glue jobs.
- Catalog data and get a unified view with the AWS Glue Data Catalog, which stores metadata including schema information about data sources and targets of your ETL jobs. When ingesting multiple data sources, the Data Catalog provides a centralized location to store and manage metadata about the various data assets across different AWS services.

- Use AWS Glue Studio to author ETL jobs that bring data in from different sources and load it into a target. AWS Glue Studio provides an easy-to-use graphical interface to author ETL jobs. With AWS Glue Studio, you can pull data from Amazon S3 or other sources, configure a transformation that joins or transforms source data, specify a target location for the transformed data, view the schema or a sample of the dataset, and run, monitor, and manage the ETL jobs.
- Perform serverless ETL processing in the AWS Glue Spark runtime engine, which is a serverless compute engine for running ETL jobs. Designed to be highly scalable to process large volumes of data quickly and efficiently, the AWS Glue runtime engine supports both Apache Spark-based jobs, which are heavy-duty and so for large datasets, and Python shell-based jobs, which are lightweight and so for smaller datasets or simple transformations.
- Orchestrate complex ETL tasks with interdependencies by using workflows.

After AWS Glue detects the data from the sources and generates metadata in the target, we can use AWS Athena to query and analyze. Let's look at the Athena service.

Amazon Athena

Amazon Athena is a serverless, interactive, query-managed service that allows users to analyze data stored in Amazon S3 by using standard SQL queries. With Athena, users can easily query data in S3 without the need to set up or manage any infrastructure. Athena is designed to work with a wide variety of data formats, including CSV, JSON, ORC, Parquet, and Avro. It also supports complex data types, such as arrays and maps, making it easy to query nested data structures. Athena has the following features:

- **Serverless architecture**: Athena is a serverless service, which means users don't need to form or manage any infrastructure. AWS takes care of all the underlying infrastructure management, including scaling, monitoring, and maintenance.
- **Standard SQL support**: Athena supports standard SQL, which makes it easy for users to get started and query data using their existing SQL skills.
- **Integration with AWS S3**: Athena integrates seamlessly with Amazon S3, making it easy to analyze data stored in S3 without the need to move the data.
- **Compatibility with popular BI tools**: Athena is compatible with a variety of popular BI tools, including Tableau, Amazon QuickSight, and Microsoft Power BI.
- **Cost-effective**: The service is cost-effective since users only pay for the queries they run.

Now, let's get our hands dirty and provision an AWS Glue service to ingest data from a CSV spreadsheet, and then use the Amazon Athena service to analyze the data with SQL queries:

1. Use S3 to set up **the source and target for the Glue service**, as in the following screenshot:

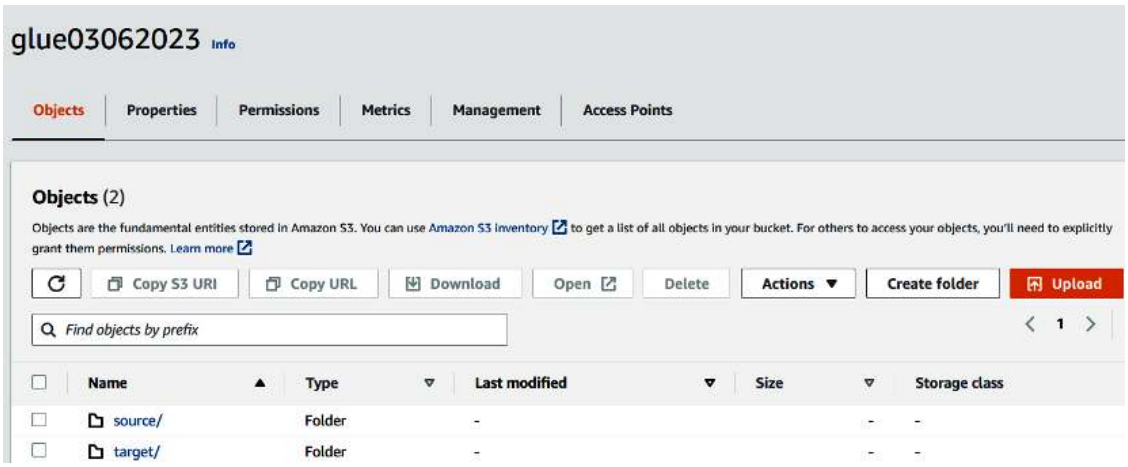


Figure 5.1 – Glue data ingestion source and target store

As shown in *Figure 5.1*, we have created an S3 bucket named **Glue03062023** and two folders underneath – **source**, which has the source data file uploaded from a client, and **target**, which will be the output destination of the AWS Glue crawler job.

2. Use IAM to create a **role for Glue:**

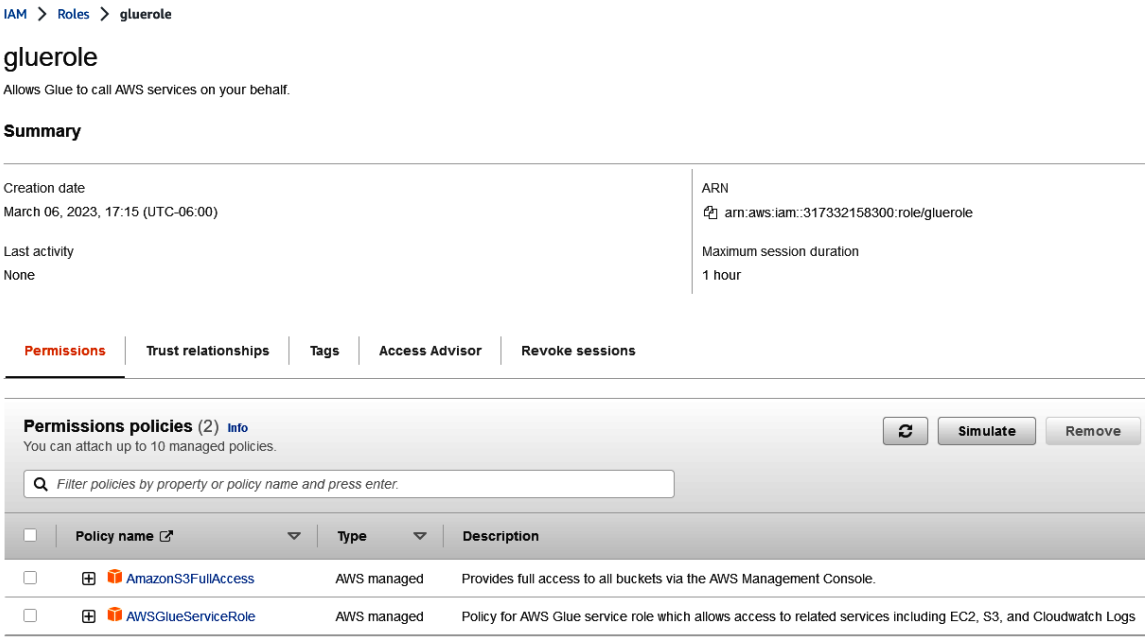


Figure 5.2 – IAM gluerole role in AWS Glue

As shown in *Figure 5.2*, we have created an AWS service role named **gluerole**, which is assigned the **AmazonS3FullAccess** and **AWSGlueServiceRole** permissions.

3. **Create a Glue database:**

   Log in to the AWS Management Console, then in the search box next to **Services**, search for and choose **AWS Glue** to open the AWS Glue console.

   In the navigation pane, under **Databases**, add a database named `salesdb`, as shown in *Figure 5.3*:

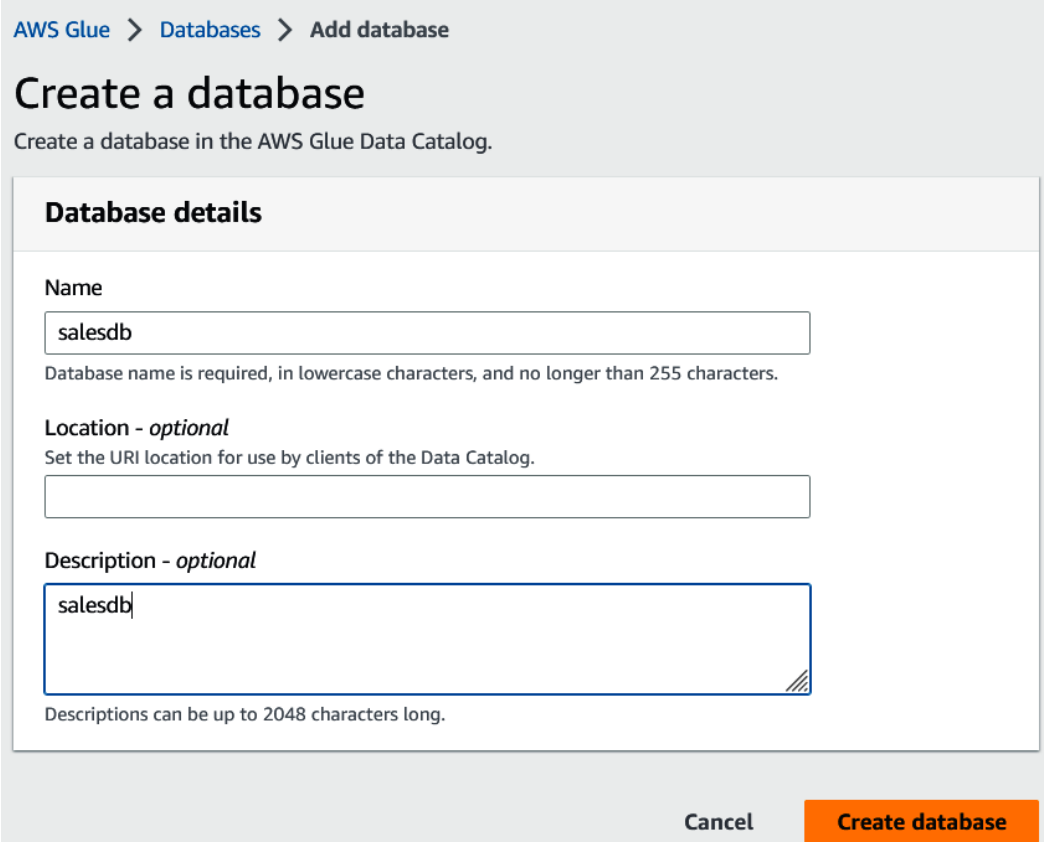


Figure 5.3 – Create an AWS Glue database called salesdb

4. **Configure and create the AWS Glue crawler:**

Under the created database, select **Tables**. Then, click **Add tables using crawler**.

For **Name**, enter `sales`. Click **Next** at the bottom of the page. Select **Add a data source** and configure the following, as shown in *Figure 5.4*:

- **Data sources**: Choose **S3**
- **Location of S3 data**: Choose **in the same account**, and enter the source S3 bucket: `s3://glue03062023/source/`
- **Existing IAM role**: Choose **gluerole**, which we created earlier

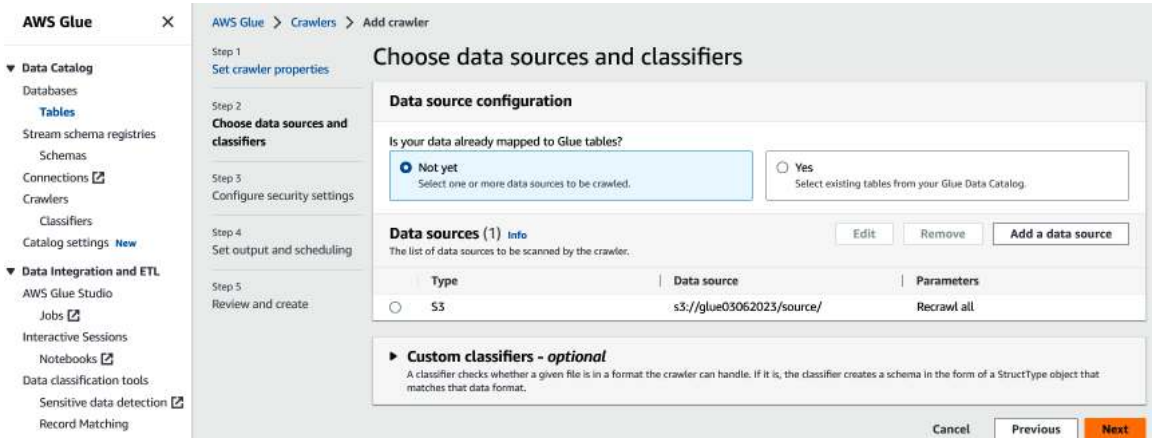


Figure 5.4 – Choose data sources for the Glue crawler

Click **Next**. In the **Output configuration** section, select **Add database**. For **Name**, enter `salesdb`. In the **Crawler schedule** section, for **Frequency**, keep the default **On demand**, as shown in *Figure 5.5*:

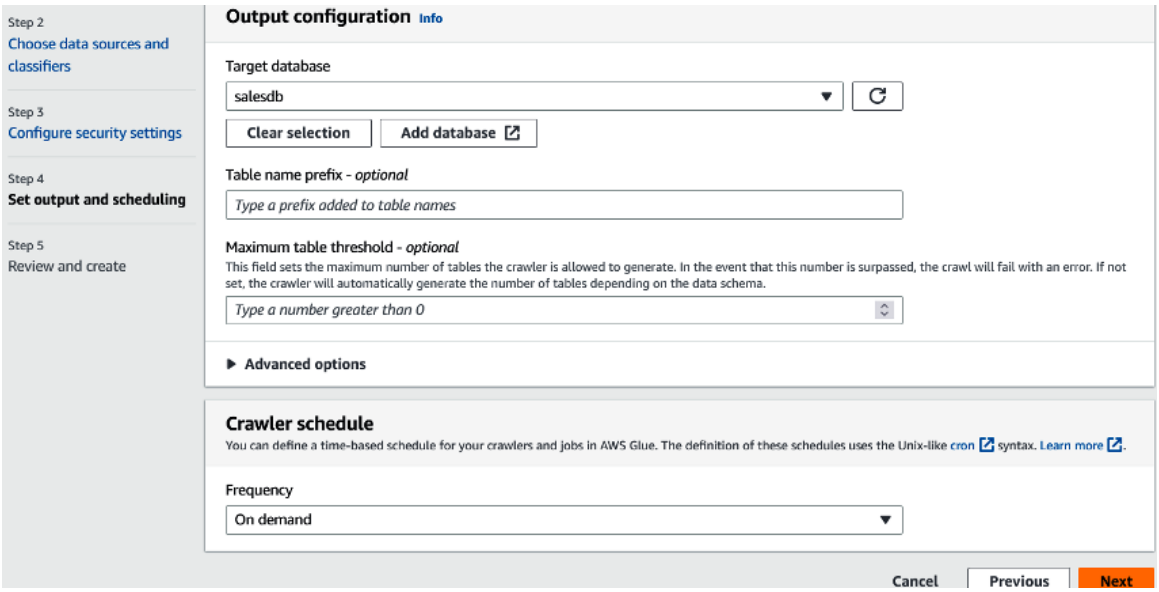


Figure 5.5 – Crawler output configuration

Click **Next** to go to the **Review and create crawler** page, as shown in *Figure 5.6*. Click **Create crawler**:

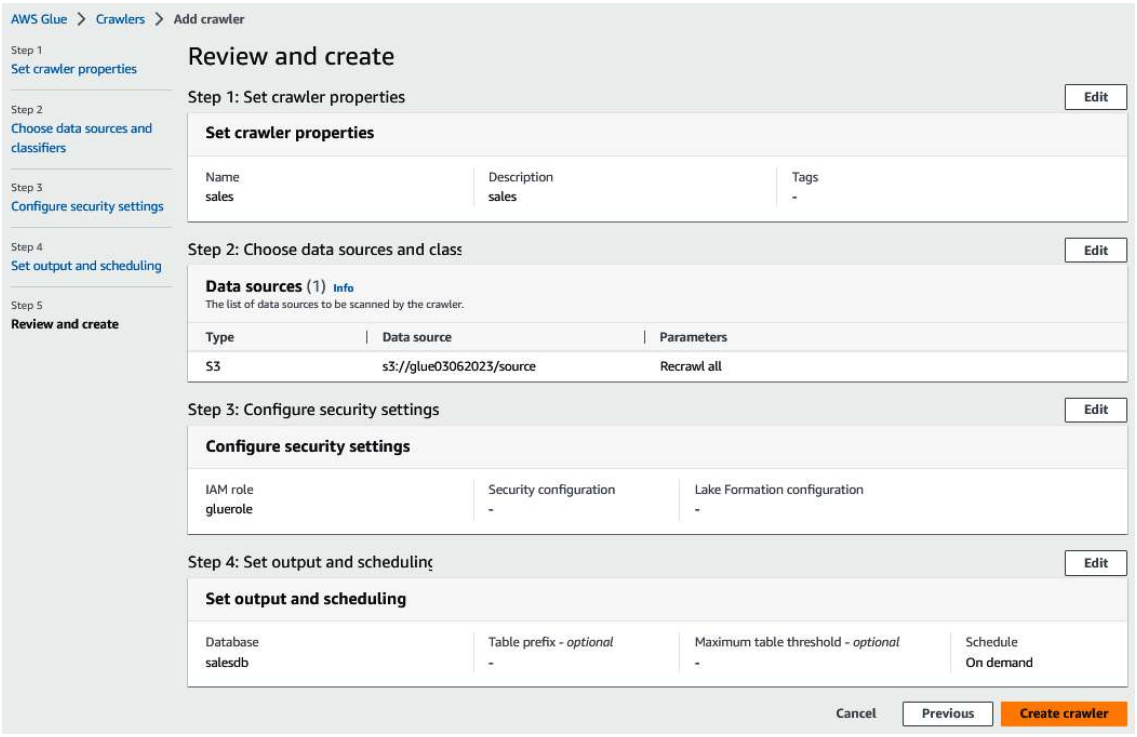


Figure 5.6 – Review and create crawler

5. **Run the crawler:**

After the crawler is created, select it and click **Run**.

The crawler state changes to **Running**, then to **Completed**, as shown in *Figure 5.7*:

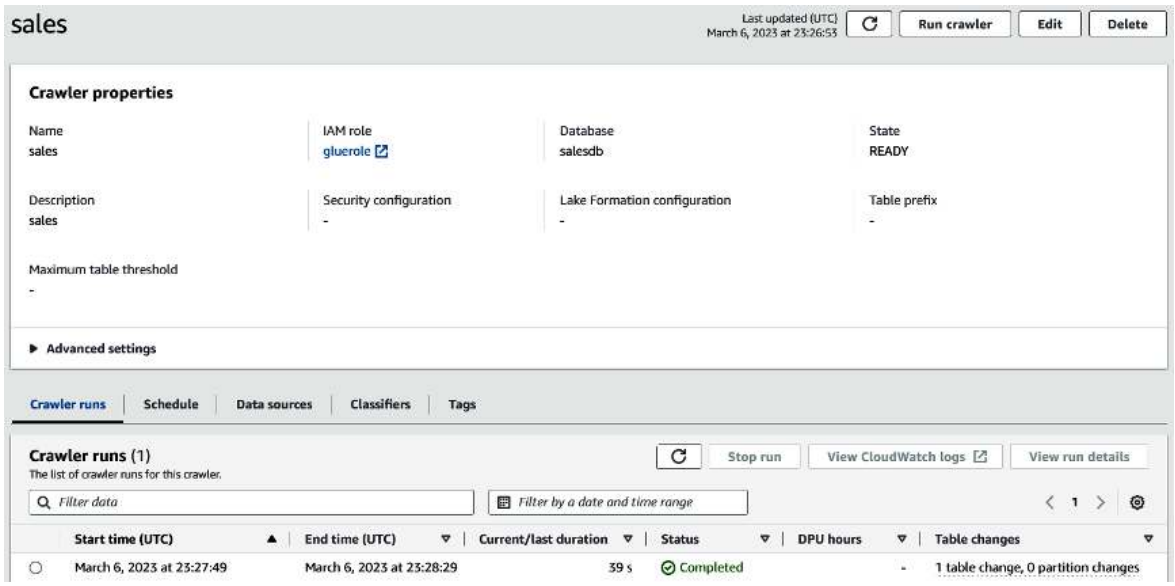


Figure 5.7 – Crawler job was run and completed

6. **Review the metadata that AWS Glue created:**

   In the navigation pane, select **Databases**. Choose the link for the **salesdb** database. In the **Tables** section, click the **sales** link.

   Review the metadata that the crawler has captured, as shown in *Figure 5.8*. The schema lists the columns that the crawler discovered from the source dataset.

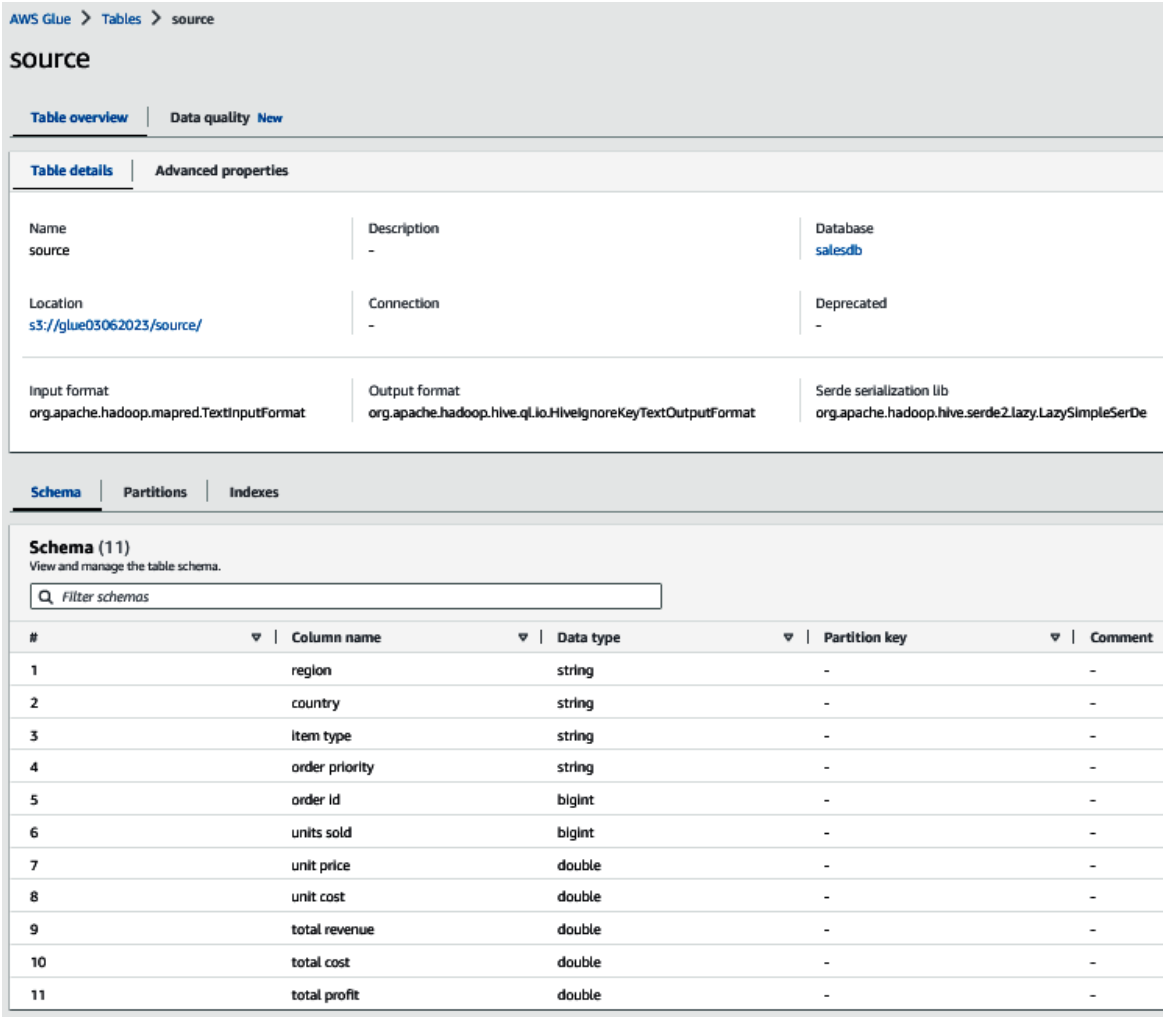
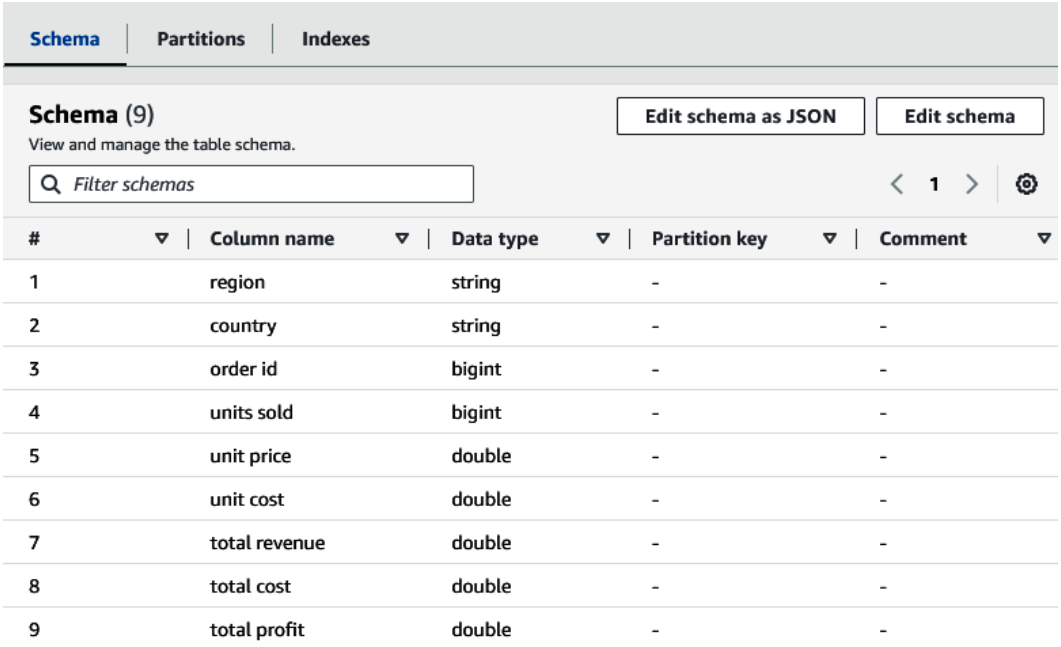


Figure 5.8 – Metadata crawler created

7. **Edit the data schema:**

From the **Actions** menu in the upper-right corner of the page, choose **Edit schema**. Change the schema as needed and click **Update schema**. *Figure 5.9* shows the updated schema:



#	Column name	Data type	Partition key	Comment
1	region	string	-	-
2	country	string	-	-
3	order id	bigint	-	-
4	units sold	bigint	-	-
5	unit price	double	-	-
6	unit cost	double	-	-
7	total revenue	double	-	-
8	total cost	double	-	-
9	total profit	double	-	-

Figure 5.9 – Updated schema

8. **Configure an S3 bucket to store Athena query results**:

In the Glue navigation pane, under **Databases**, choose **Tables**. Choose the link for the sales table.

Select **Actions | View data**. When the popup appears to warn you that you will be taken to the Athena console, click **Proceed**.

The Athena console opens. Before you run a query in Athena, you need to specify an S3 bucket to hold the query results.

Click on the **Settings** tab. Choose **Manage**.

To the right of **Location of query result**, click on **Browse S3**. Choose the bucket name **s3://glue03062023/target**. Select **Choose**. Fill in the **Expected bucket owner** section and then click **Save**, as shown in *Figure 5.10*:

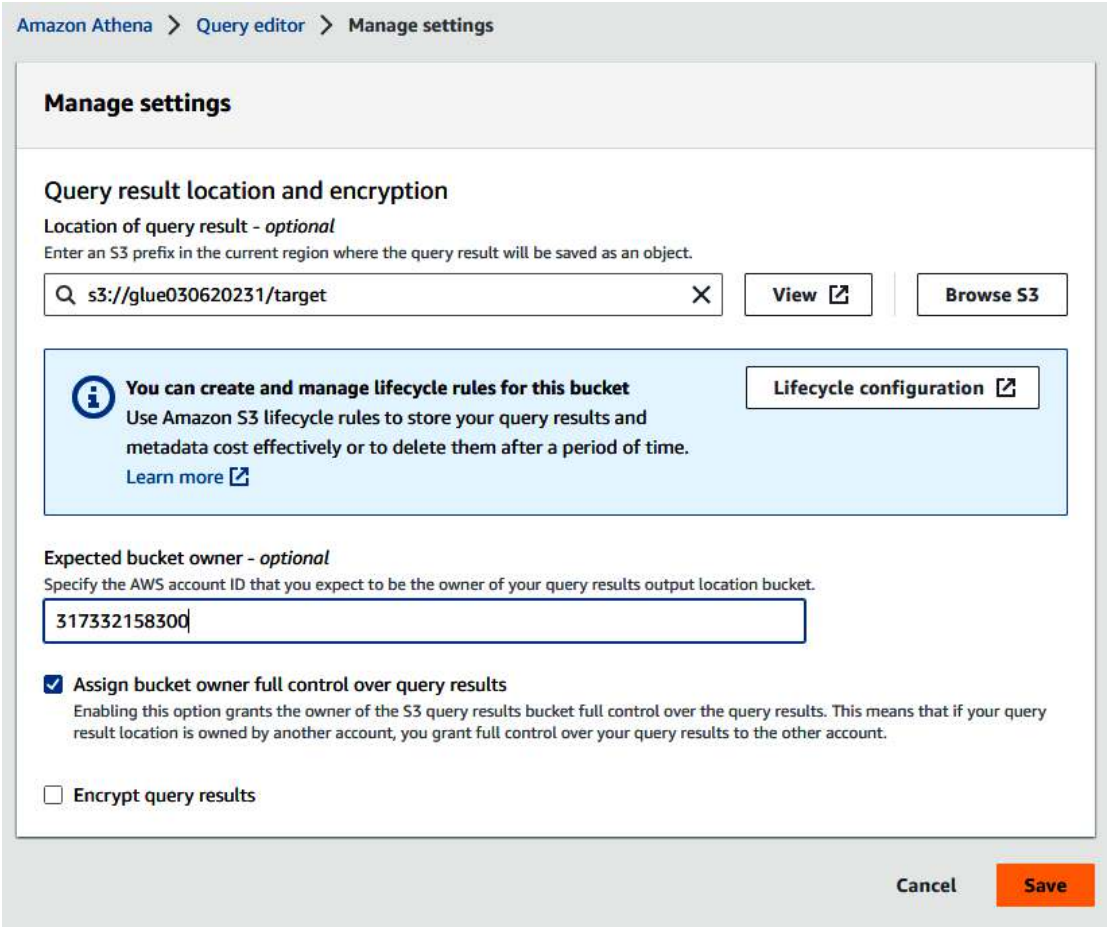


Figure 5.10 – Athena query result settings

9. **Preview a table in Athena**:

Return to the Athena query editor.

Click on the **Editor** tab. In the **Data** panel on the left, notice that the data source is **AwsDataCatalog**. For **Database**, choose **salesdb**.

In the **Tables** section, click the ellipsis (three-dot) icon for the **sales** table, and then select **Preview Table**. The first 10 records from the **sales** table will display.

10. **Create a table for the Europe region**:

In the Athena query editor, copy and paste the following query into a `query` tab:

```
CREATE table salesdb.region
WITH (
```

```
  format='PARQUET', external_location='s3://glue030620231/
target/'
) AS SELECT * FROM source
where region = 'Europe';
```

Click **Run**. The query and result are shown in *Figure 5.11*:

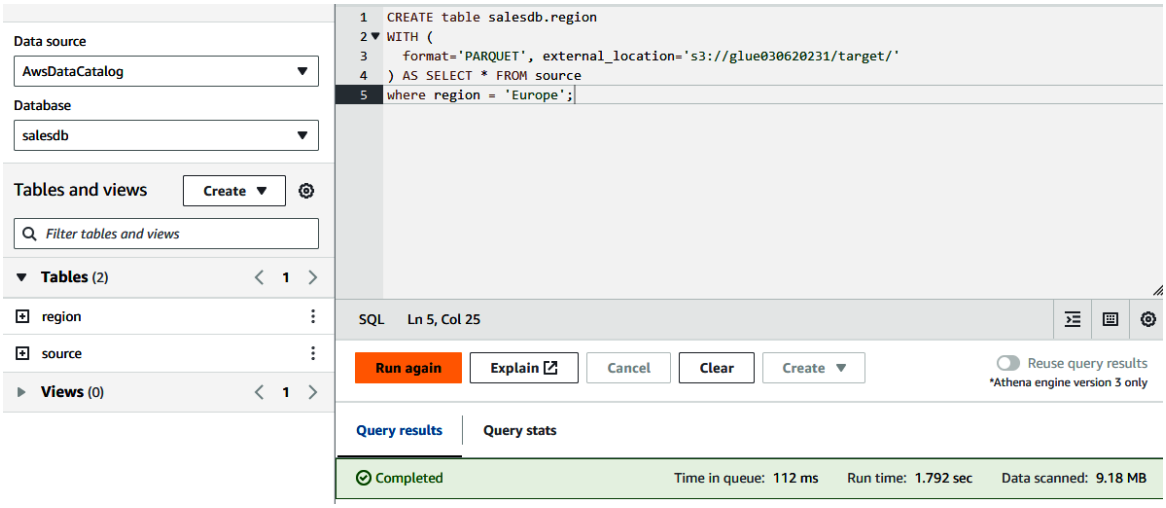


Figure 5.11 – Athena query and result

To preview the results, in the **Tables** section, to the right of the **region** table, click the ellipsis icon, and then select **Preview Table**.

Now that you have isolated the data that you are interested in, you can write queries for further analysis.

Through the preceding process, you learned how to use Athena to query tables in a database that an AWS Glue crawler created. You built a table called `salesdb.region` from the original dataset and store output to the target S3 bucket.

As we can see, AWS Glue created a crawler to ingest the original dataset from the S3 target folder into a database and infer the appropriate schema, which can be edited. Then, using Athena, we can develop queries and better understand the data. This integration reduces the time that it takes to derive insights from the raw data and apply these insights to make better decisions. With AWS Glue, we can batch-ingest data from various data sources.

In the next section, we will switch gears and discuss streaming data ingestion with Amazon Kinesis.

The Amazon Kinesis family

The **Amazon Kinesis** family is a set of fully managed services provided by AWS for streaming data processing and analysis. The family consists of the following main services:

- **Amazon Kinesis Data Streams**: This is a service for collecting and processing large amounts of data in real time from various sources, such as websites, mobile apps, IoT devices, and social media. Data is stored in shards and processed with custom applications. You can use Kinesis Data Streams to process data with Amazon Lambda and other custom applications. Kinesis Data Streams also offers the ability to store data in Amazon S3, enabling you to perform additional analysis on data stored in Amazon S3.
- **Amazon Kinesis Data Firehose**: This is a fully managed service that enables you to capture, transform, and load streaming data into various destinations. Firehose provides a simple and scalable way to capture and transform streaming data from various sources, such as IoT devices, clickstreams, social media feeds, and server logs. Kinesis Data Firehose supports a variety of destination options, including Amazon S3 for cost-effective and durable storage of raw data, Amazon Redshift for data warehousing and analytics, and Amazon Elasticsearch for full-text search and analysis. Kinesis Data Firehose also supports third-party destinations such as Splunk, Datadog, and New Relic.
- **Amazon Kinesis Data Analytics**: This is a fully managed service provided by AWS that enables you to perform real-time data analysis on streaming data using SQL queries. Kinesis Data Analytics provides a simple and scalable way to perform real-time data analysis on streaming data from various sources, such as Amazon Kinesis Data Streams, Amazon Kinesis Data Firehose, and Amazon S3. You can use Kinesis Data Analytics to query data from these sources, apply real-time transformations, and then output the results to a variety of destinations, such as Amazon S3, Amazon Kinesis Data Streams, and Amazon Elasticsearch.

The AWS Kinesis family provides powerful big data processing capacities, such as real-time analytics, machine learning, and security monitoring. *Figure 5.12* shows two paths for data streaming ingestion. In the top path, streaming data from various sources is ingested into **Kinesis Data Streams**, passed to **Kinesis Data Firehose**, and stored in **Amazon S3** for future use. In the second path, after data is ingested into **Kinesis Data Streams**, it is fed to **Kinesis Data Analytics** for real-time analytics:

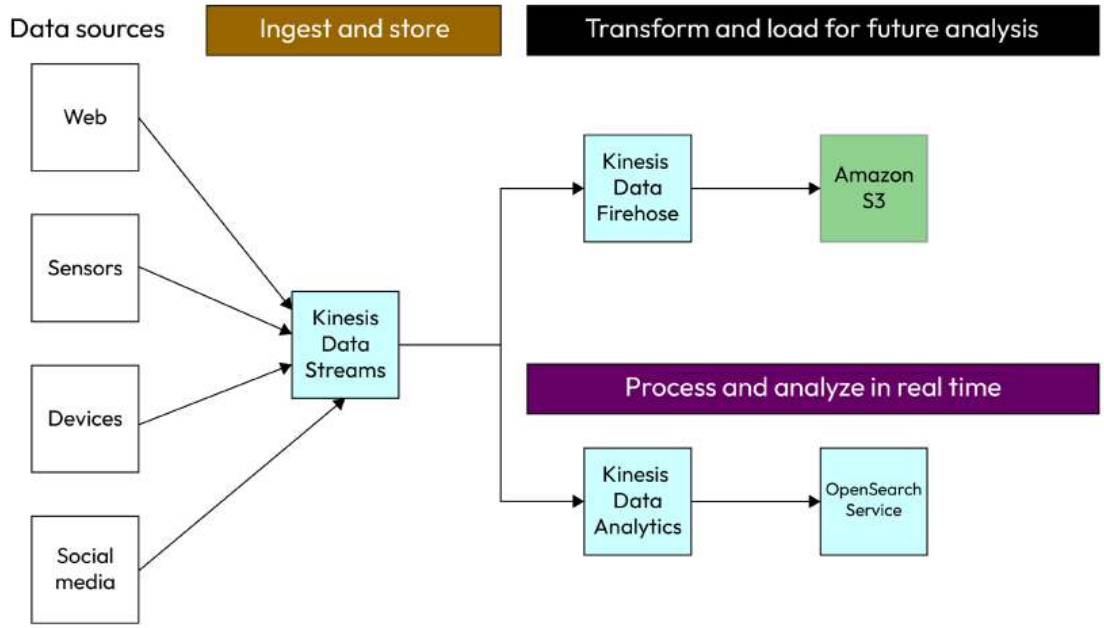


Figure 5.12 – Amazon Kinesis family

As shown in *Figure 5.13*, Kinesis Data Firehose can ingest streaming data and store it in an Amazon cloud storage service such as S3, Redshift, or OpenSearch:

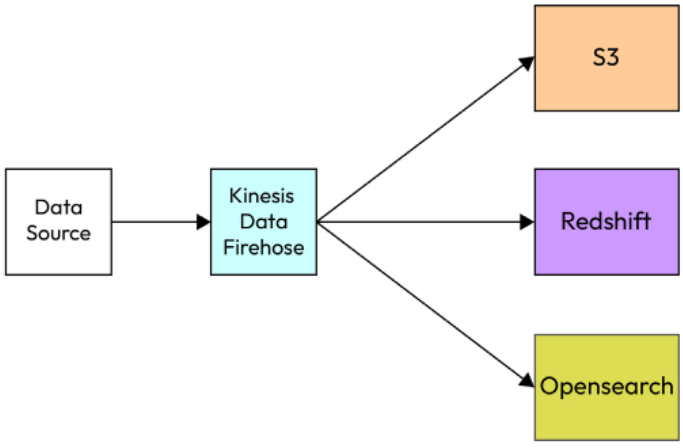


Figure 5.13 – Amazon Kinesis DataFirehose

After introducing the concepts, now let us show how to set Kinesis Firehose up to ingest data and store outputs to Amazon S3 buckets:

1. **Create a target for Kinesis Firehose:**

   Create a target S3 bucket for Firehose to store raw data, as shown in *Figure 5.14*:

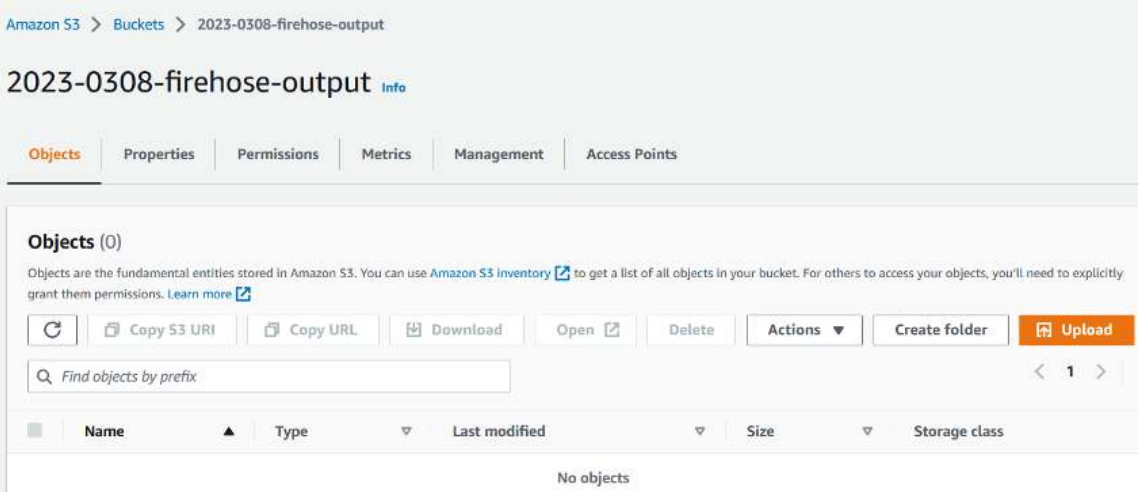


Figure 5.14 – Create a target S3 bucket

2. **Create a Kinesis delivery stream:**

Log in to the AWS console and navigate to **Kinesis**. Click **Create delivery stream**, then fill in the details as shown in *Figure 5.15*.

> **Note**
>
> We have used **Direct PUT** as the source – it means Kinesis Data Firehose will ingest data directly into the source. Also, note that we have used 2 MiB or 120 seconds as the buffer size, whichever comes first, and we have used all the default settings.

Click **Create delivery stream**:

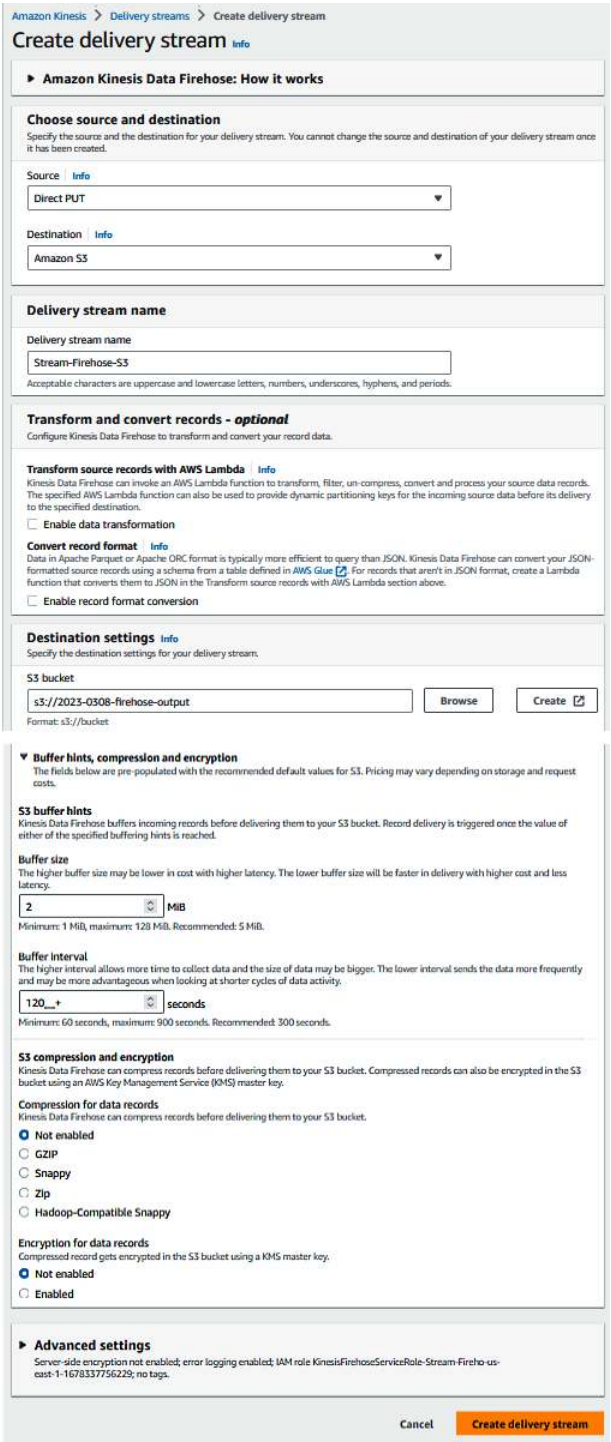


Figure 5.15 – Firehose delivery stream details

3. **Test with demo data:**

When the delivery stream shows as **Active**, click its name, **Stream-Firehose-S3**, and then click **Test with demo data**, as shown in *Figure 5.16*. This test runs a script and puts demo data in your Kinesis Data Firehose delivery stream. Click **Start sending demo data** and wait for several minutes, then click **Stop sending demo data**:

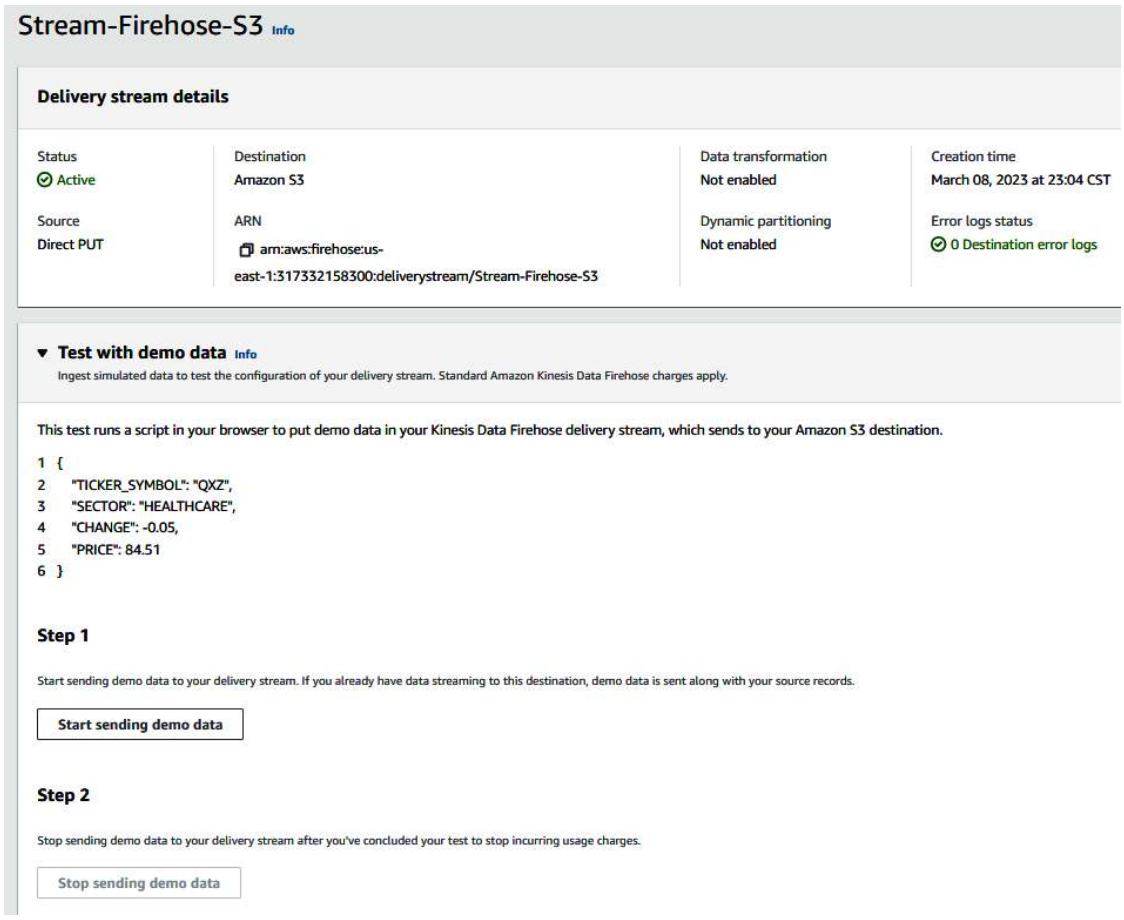


Figure 5.16 – Test with demo data

4. **Examine the target output:**

Now, check the S3 target folder. You will see output files are generated in the target S3 bucket folder. As shown in *Figure 5.17*, there are four files generated in the target:

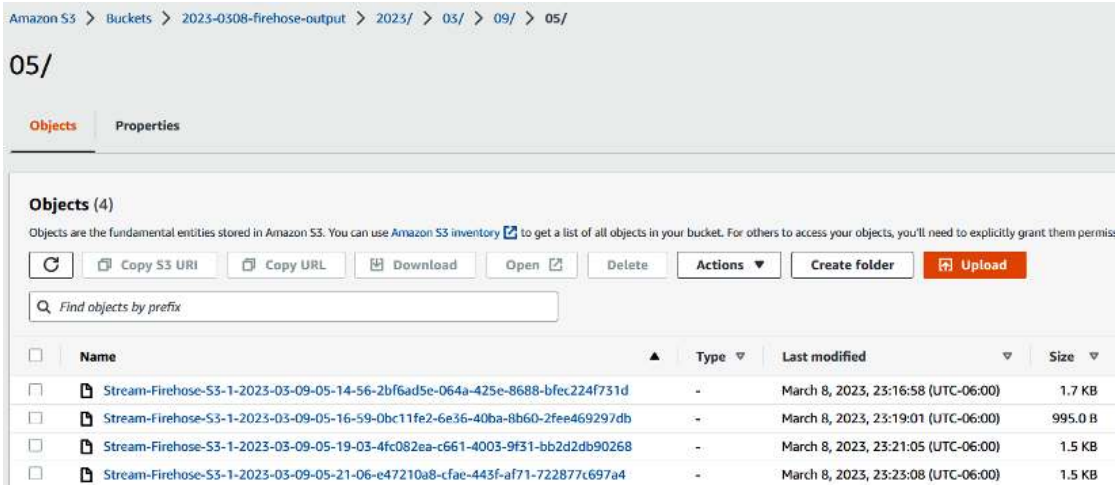


Figure 5.17 – Target output files

With the preceding steps, we have used Kinesis Firehose to directly ingest the demo data and output it into the target S3 folder. In the next section, we will introduce Amazon QuickSight and use it to visualize these output files.

Amazon QuickSight

Amazon QuickSight is a cloud-based BI and data visualization service. It enables users to easily create interactive dashboards, perform ad hoc data analysis, and share insights with others in their organization. Some of the key features of Amazon QuickSight include the following:

- **Data connectivity**: Amazon QuickSight can connect to a wide range of data sources, including AWS services such as Amazon S3, Amazon Redshift, and Amazon RDS, as well as other popular data sources, such as Salesforce, MySQL, and Microsoft Excel.
- **Data preparation**: Amazon QuickSight provides a simple, intuitive interface for preparing data for analysis, including features such as data cleaning, filtering, and aggregation.
- **Data visualization**: Amazon QuickSight offers a variety of visualization options, including charts, tables, and maps, allowing users to easily create interactive dashboards and reports.
- **Team collaboration**: Amazon QuickSight allows users to share dashboards and reports with others in their organization and provides access controls to ensure that data is shared only with authorized users.
- **Integration with other AWS services**: As a virtualization tool, QuickSight integrates seamlessly with the other Amazon data analytics services.
- **Leveraging Super-fast, Parallel, In-memory Calculation Engine (SPICE)**: This is an in-memory data caching service for faster querying and visualization of data.

Amazon QuickSight can be used in Business Intelligence to create dashboards and reports for BI purposes, allowing users to easily analyze and visualize data and gain insights into business performance. It can be used in sales and marketing to analyze sales and marketing data, such as customer demographics, purchase history, and campaign performance, allowing users to optimize their sales and marketing strategies. It can be used in finance and accounting to analyze financial data, such as revenue, expenses, and cash flow, allowing users to monitor financial performance and identify areas for improvement.

Amazon QuickSight is a powerful tool for data visualization and analysis and provides a wide range of features for data connectivity, preparation, and visualization. It is a great platform for businesses to gain insights from their data and make data-driven decisions.

Now we will show how to use QuickSight to visualize the data we generated from Amazon Kinesis Firehose in the previous section:

1. **Sign up for a QuickSight account**

Log in to the AWS console, then navigate to **QuickSight**. Sign up for a QuickSight account if you haven't done so already.

Generate the QuickSight manifest file.

We will use the data file generated by Kinesis Firehose earlier. We need to find the data file URL and create a manifest file named `manifest1`, as shown in *Figure 5.18*:

```
{
    "fileLocations": [
        {
            "URIs": [
                "s3://2023-0308-firehose-output/2023/03/09/05/Stream-Firehose-S3-1-2023-03-09-05-14-56-2bf6ad5e-064a-425e-8688-bfec224f731d"
            ]
        },
    ],
    "globalUploadSettings": {
        "format": "JSON",
        "delimiter": ",",
        "textqualifier": "'",
        "containsHeader": "true"
    }
}
```

Figure 5.18 – QuickSight manifest file

2. **Define the dataset:**

Log in to your QuickSight account, click **New dataset**, and then choose **s3**. Fill in the details in the pop-up window shown in *Figure 5.19*:

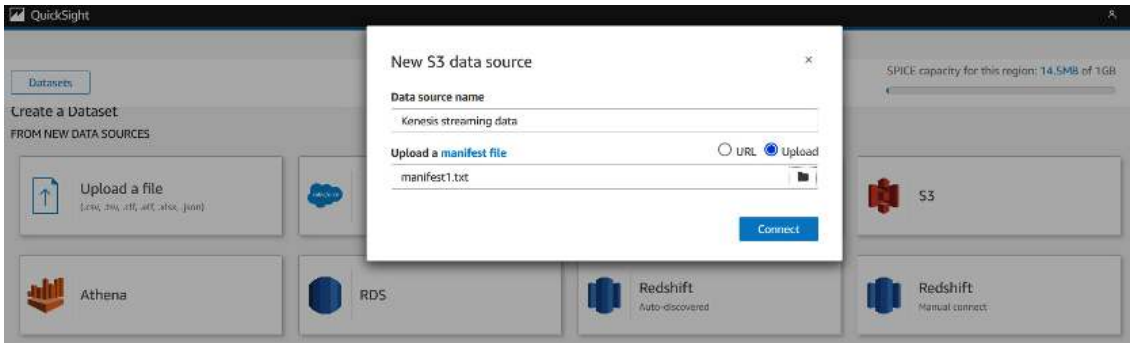


Figure 5.19 – Generate dataset

3. **Connect to the dataset:**

Click **Connect**. The new dataset is created, as shown in *Figure 5.20*. Click **Visualize**:

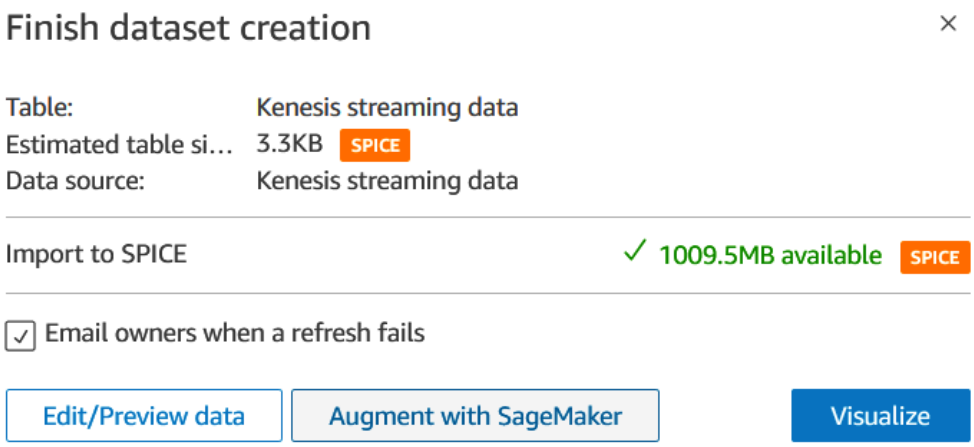


Figure 5.20 – Visualize dataset

4. **Create data visualization:**

There are four fields shown for the dataset. Choose **Bar chart** as the visual type, **Ticker symbol** as the *x* axis, and **Price(sum)** as the *y* axis. A graph will be generated, as shown in *Figure 5.21*:

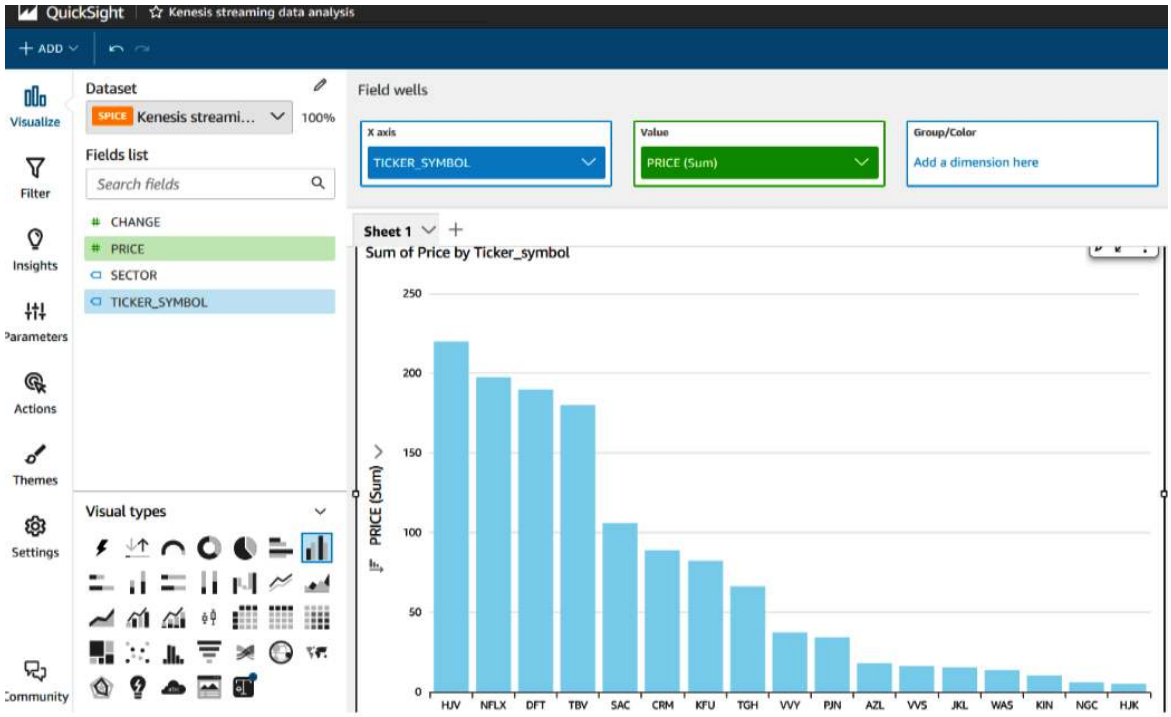


Figure 5.21 – Visualization

This concludes our hands-on exercises on data ingestion using Amazon Kinesis Firehose, storing data in S3, and visualizing data using QuickSight. In the next section, we will examine Amazon EMR, which is a powerful data processing platform.

Amazon EMR

Amazon EMR is a platform for leveraging many big data tools for data processing. We will start by looking at the concepts of MapReduce and Hadoop.

MapReduce and Hadoop

MapReduce and **Hadoop** are two related concepts in the field of distributed computing and big data processing.

The idea of MapReduce is "divide and conquer": decompose a big dataset into smaller ones to be processed in parallel on distributed computers. It was originally developed by Google for its search engine to handle the massive amounts of data generated by web crawling. The MapReduce programming model involves two functions: a map function that divides and processes in parallel the datasets and a map function that aggregates the map outputs.

Hadoop is an open source software framework that implements the MapReduce model. Hadoop consists of two core components: **Hadoop Distributed File System** (**HDFS**) and MapReduce. HDFS is a distributed filesystem that can rapidly transfer data between Hadoop cluster nodes. MapReduce can process big data on distributed nodes and scale massively.

In summary, MapReduce is a programming model that enables parallel processing of large datasets, while Hadoop is a software framework that implements MapReduce and provides a distributed filesystem for storing and managing large datasets.

AWS EMR

AWS EMR is a fully managed cloud service that allows you to easily process large amounts of data using open source tools such as Apache Hadoop, Spark, Hive, Presto, and Flink. With EMR, you can quickly and easily create and configure a Hadoop cluster on AWS infrastructure, allowing you to perform big data processing tasks without the need for upfront hardware investment or management. EMR also provides several useful features, including the following:

- **Pre-installed applications**: EMR comes pre-installed with many popular big data processing tools, allowing you to get started quickly
- **Autoscaling**: EMR automatically scales the cluster based on the workload, allowing you to efficiently use resources and reduce costs
- **Integration with AWS services**: EMR integrates with other AWS services, such as S3, EC2, and CloudWatch, making it easy to move data into and out of the cluster and monitor its performance
- **Security**: EMR provides several security features, including encryption at rest and in transit, secure network configuration, and integration with AWS IAM
- **Storage**: EMR provides different types of storage options, including HDFS and **EMR File System** (**EMRFS**, which extends HDFS to S3)

EMR is a powerful and flexible tool that allows you to easily process large amounts of data using open source tools on AWS infrastructure. The components of an EMR cluster can be classified into four categories: core components, optional components, data sources, and external interfaces.

EMR core components are the main components of an EMR cluster. They include the following:

- **Master nodes**, which act as the entry point to the cluster and manage HDFS, job scheduling, and coordination with other cluster nodes.
- **Core nodes**, which provide compute and storage resources for processing big data jobs. They store data in HDFS, which is the distributed filesystem of the cluster.
- **Task nodes**, which provide additional compute resources to run parallelized tasks in the cluster. Task nodes do not store data in HDFS and are used to offload processing from the core nodes. For cost saving, spot instances can be used for task nodes.

EMR optional components can be added to the cluster based on the use case. Example optional components are the following:

- **Apache HBase**: A distributed, scalable, and NoSQL database system on top of Hadoop
- **Hive**: A data warehousing and SQL-like query engine for Hadoop
- **Pig**: A high-level language for querying large datasets
- **Hue**: A web-based user interface for interacting with Hadoop services
- **Presto**: A distributed SQL query engine designed for interactive analytics
- **Flink**: A real-time streaming data processing framework
- **Spark**: An in-memory big data processing framework

Let's examine two optional EMR components: Apache Hive and Apache Spark.

Apache Hive

Apache Hive is a data warehousing and SQL-like query engine built on top of Hadoop. It provides a high-level abstraction of HDFS and MapReduce, allowing users to write SQL-like queries to analyze large datasets stored in HDFS. Hive is designed to make it easier for analysts and data scientists to work with large datasets without having to write low-level MapReduce code. It supports various data formats, including CSV, JSON, Avro, Parquet, and ORC. Some of the key features of Hive include the following:

- **Schema-on-read**: Hive supports schema-on-read, which means that it can read and interpret data stored in different formats
- **Partitioning**: Hive allows users to partition data based on specific columns, making it easy to query large datasets
- **Data processing**: Hive supports a wide range of data processing functions, such as filtering, sorting, joining, and aggregating
- **User-defined functions** (**UDFs**): Hive allows users to write custom UDFs to perform complex data processing tasks

Hive is a powerful tool for data analysis and processing, especially for SQL-literate users who are familiar with the SQL language. Hive enables users to work with large datasets stored in Hadoop, using a SQL-like language, which makes it an ideal tool for data warehousing and data analysis tasks.

Apache Spark

Apache Spark is an open source big data processing engine designed to perform fast, in-memory data processing. It is built on top of Hadoop and uses HDFS for distributed storage. Spark provides a programming model called **Resilient Distributed Dataset** (**RDD**), which allows developers to perform distributed data processing in a fault-tolerant manner. RDD is an immutable distributed collection of objects that can be processed in parallel across a cluster of machines. Spark provides a wide range of libraries and APIs to perform various data processing tasks, including the following:

- **Spark Core**: Provides the basic functionality of Spark, including RDD and distributed task scheduling
- **Spark SQL**: A module for structured data processing
- **Spark Streaming**: A module for the real-time processing of streaming data
- **MLlib**: A library for machine learning and data mining tasks
- **GraphX**: A library for processing graph data

Some of the key features of Spark include the following:

- **In-memory processing**: Spark uses in-memory processing to speed up data processing tasks
- **Fault tolerance**: Spark provides fault tolerance through RDD, which can be reconstructed if a node fails
- **Scalability**: Spark can scale to thousands of nodes in a cluster, making it suitable for processing large datasets
- **Compatibility**: Spark can run on Hadoop YARN, Mesos, and standalone clusters

Apache Spark is a powerful big data processing engine that provides fast, in-memory processing of large datasets. It provides a wide range of libraries and APIs to perform various data processing tasks, making it an ideal tool for big data analytics, machine learning, and real-time processing of streaming data.

Now we will create an EMR cluster and submit a workload to an Amazon EMR cluster:

1. **Launch an EMR cluster**:

Log in to the AWS console and select the EMR service. From the left navigation pane, choose **Clusters | Create Cluster | Advanced Options**.

First, fill in the **Software and Steps** section, as shown in *Figure 5.22*:

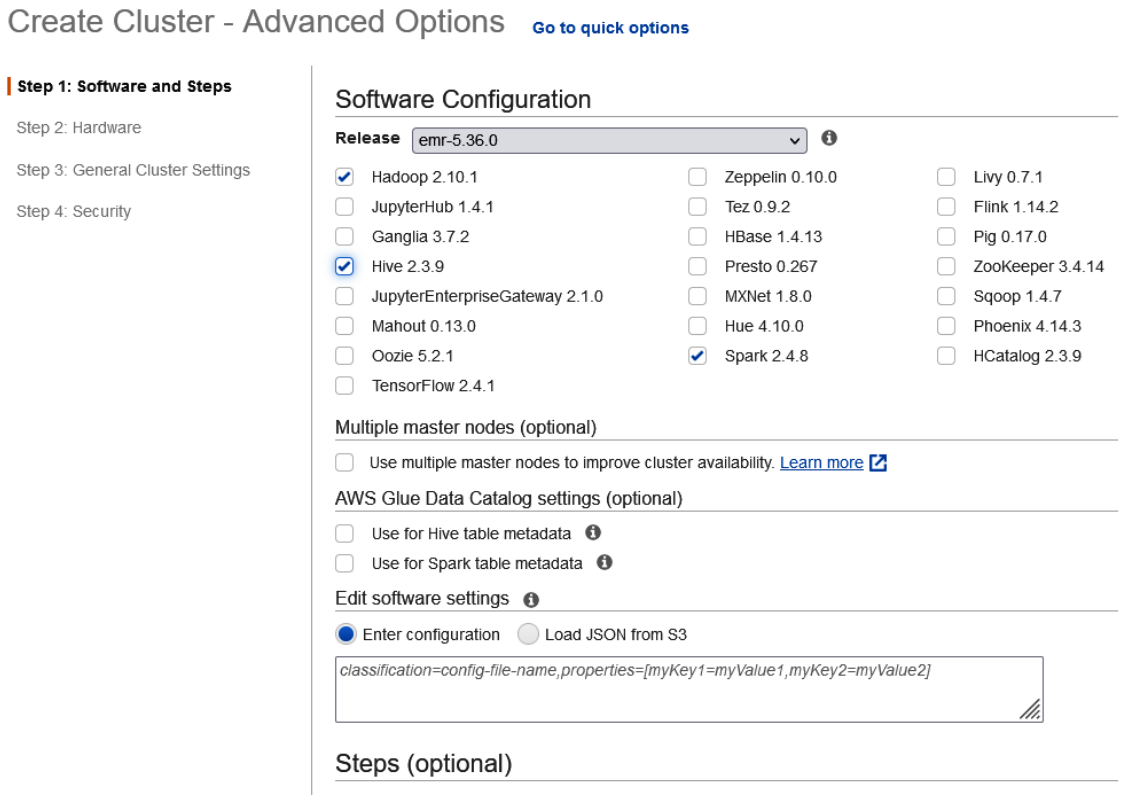


Figure 5.22 – EMR cluster creation step 1

Then, move on to step 2, which is filling in the hardware details, as shown in *Figure 5.23*:

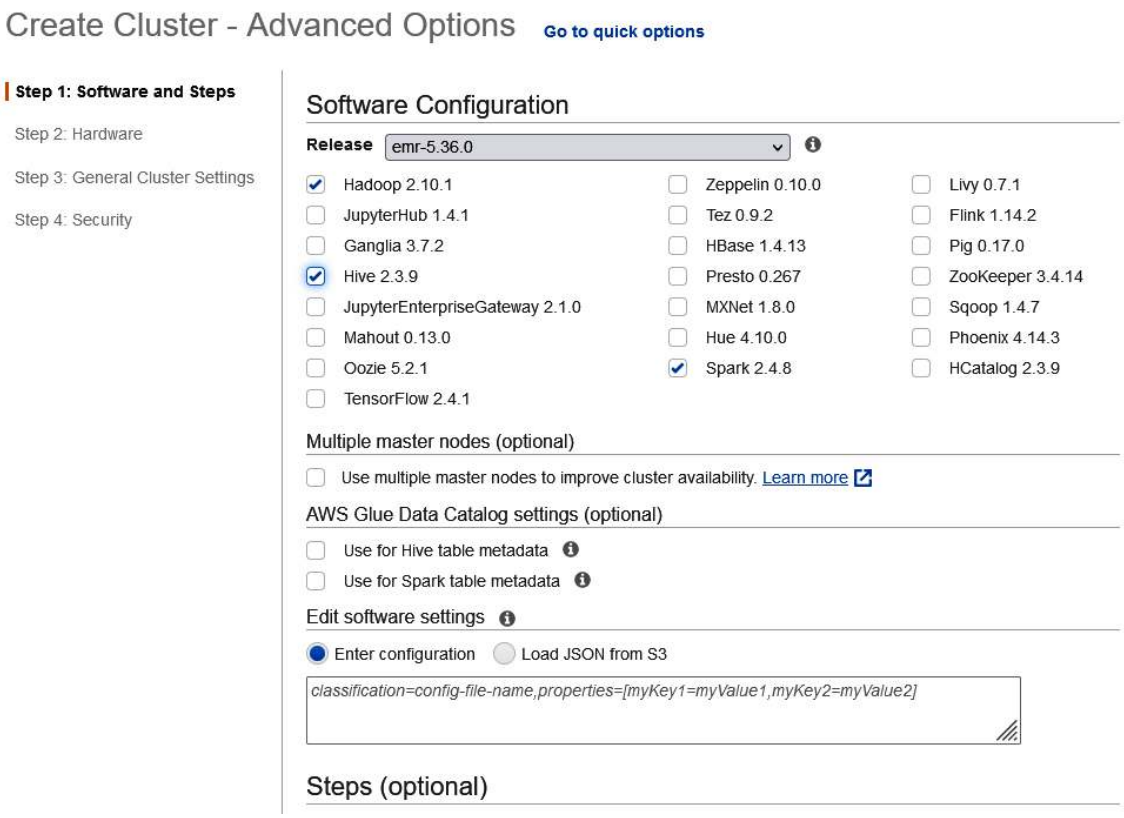


Figure 5.23 – EMR cluster creation step 2

For the third step, fill in the general cluster settings, as shown in *Figure 5.24*:

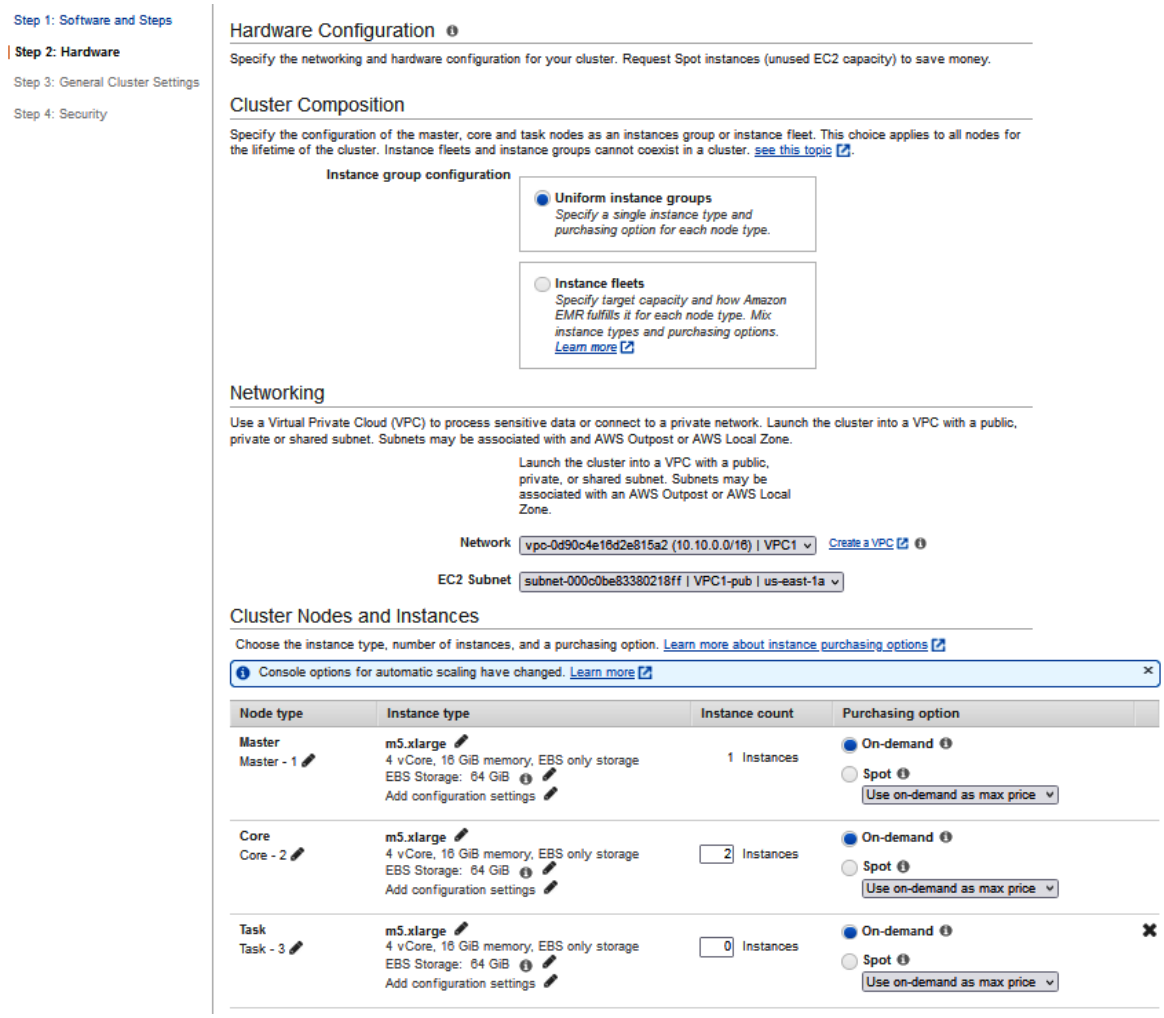


Figure 5.24 – EMR cluster creation step 3

For the fourth step, fill in the security options, as shown in *Figure 5.25*:

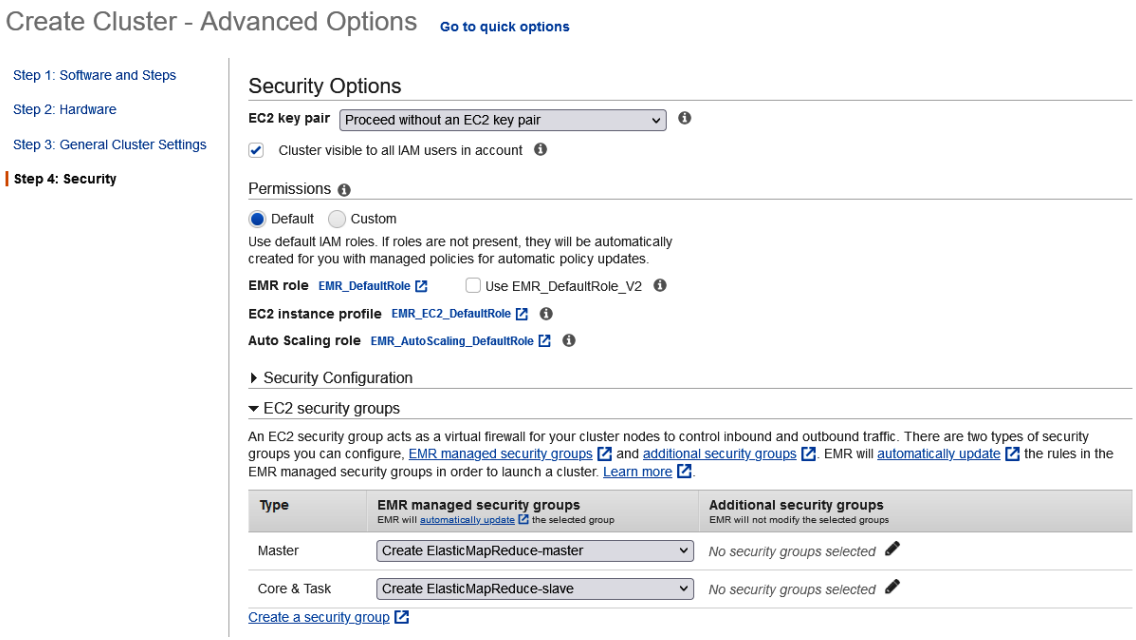


Figure 5.25 – EMR cluster creation step 4

Create the EMR cluster. It will show the state as **Starting**, as shown in *Figure 5.26*:

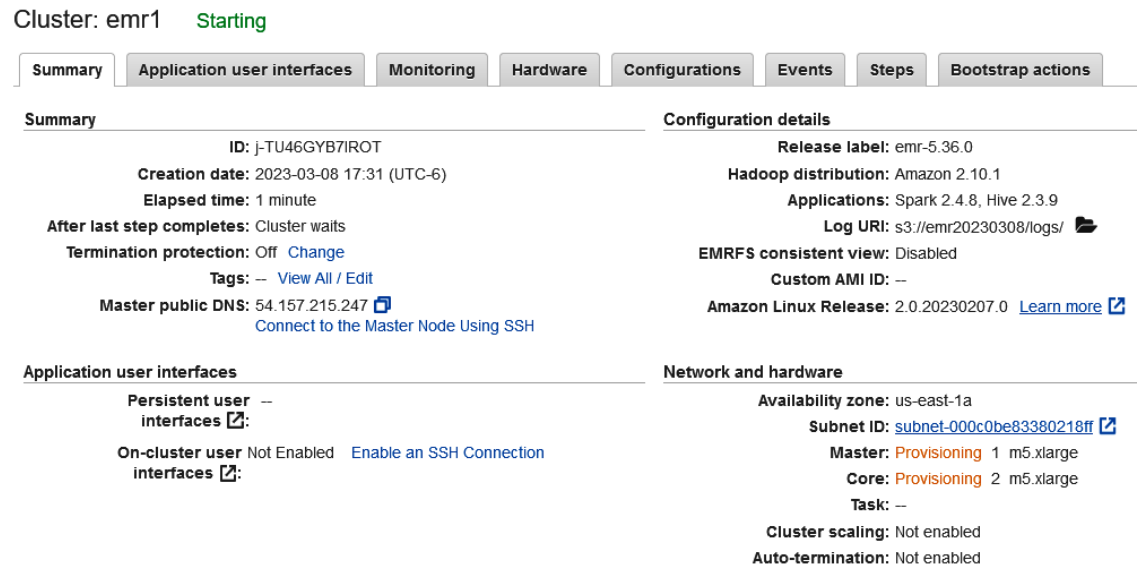


Figure 5.26 – EMR cluster Starting state

The nodes (EC2 instances) and S3 buckets will be created, as shown in *Figure 5.27*:

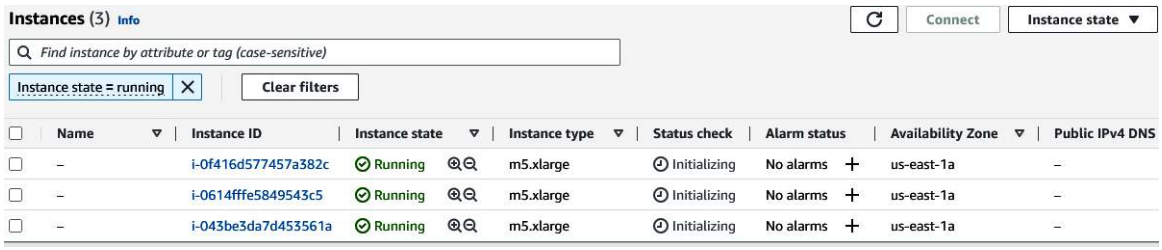


Figure 5.27 – EMR cluster nodes

The S3 buckets are created as the output locations for the cluster, as shown in *Figure 5.28*.

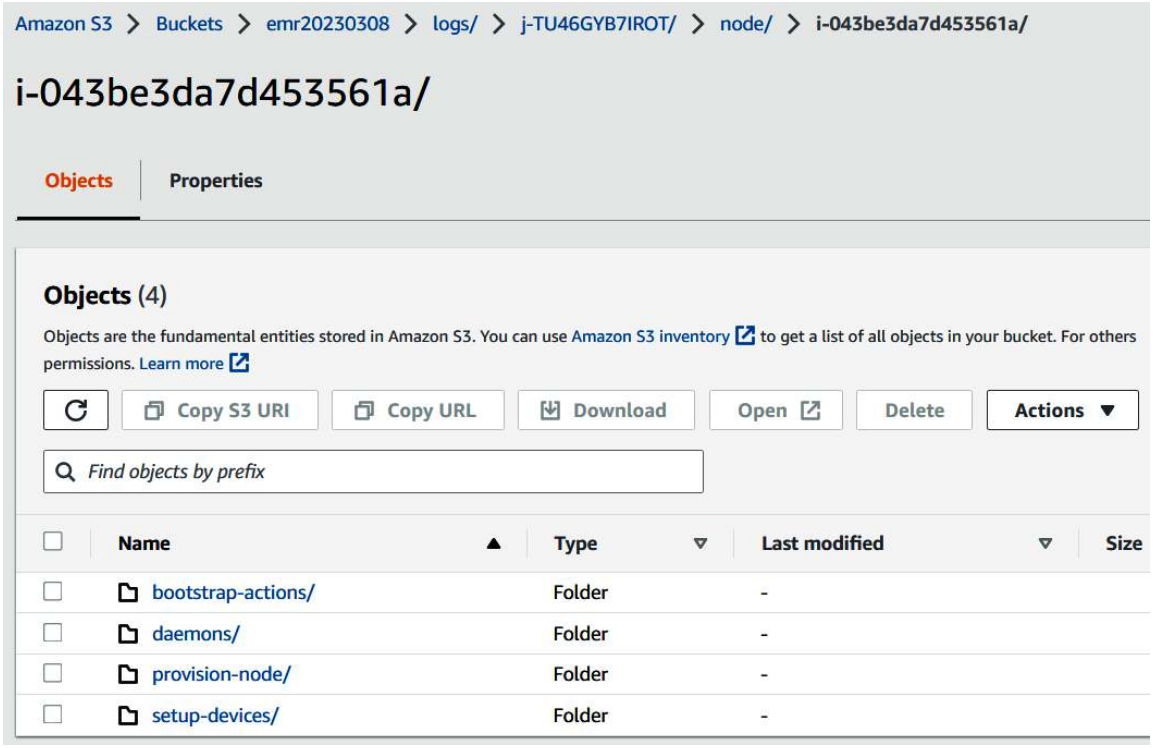


Figure 5.28 – EMR cluster S3 buckets

The cluster status changes from **Starting** to **Waiting** when the cluster is up, running, and ready to accept work, as shown in *Figure 5.29*:

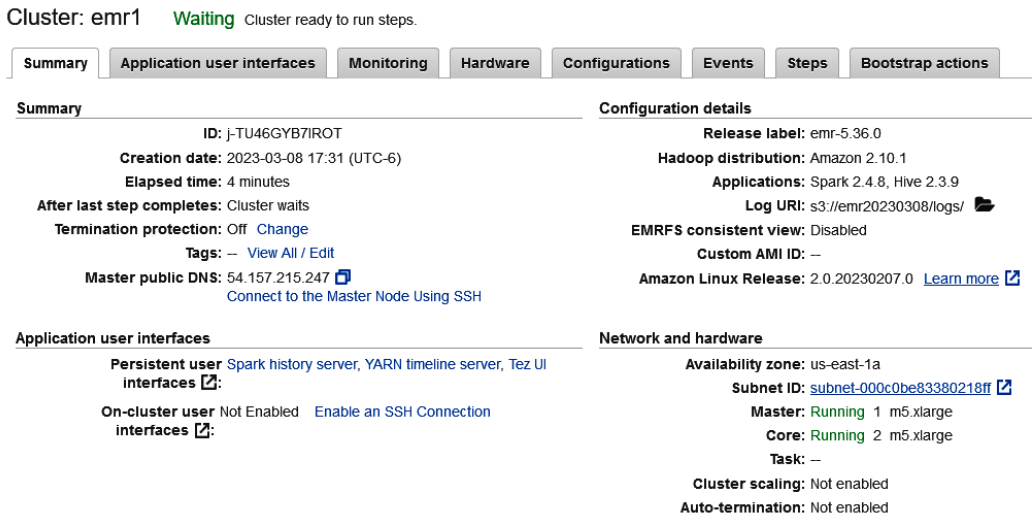


Figure 5.29 – EMR cluster in the Waiting state

2. **Submit Spark work to process food inspection data**

We will now use Apache Spark in the EMR cluster to process a big data CSV spreadsheet called `food_inspection_data.csv`, which has 200,000 rows. We will find the top 10 names that have the most violations. The table columns are shown in *Figure 5.30*:

name	inspection_result	inspection_closed_business	violation_type	violation_points
100 LB CLAM	Incomplete	FALSE		0
100 LB CLAM	Unsatisfactor	FALSE	BLUE	5
100 LB CLAM	Unsatisfactor	FALSE	RED	5
100 LB CLAM	Unsatisfactor	FALSE	RED	10
100 LB CLAM	Unsatisfactor	FALSE	RED	5
100 LB CLAM	Complete	FALSE		0
100 LB CLAM	Complete	FALSE		0
100 PERCENT NUTRICION	Unsatisfactor	FALSE	BLUE	5
100 PERCENT NUTRICION	Unsatisfactor	FALSE	BLUE	5
100 PERCENT NUTRICION	Unsatisfactor	FALSE	RED	10
100 PERCENT NUTRICION	Unsatisfactor	FALSE	RED	5
1000 SPIRITS	Satisfactory	FALSE	BLUE	5
1000 SPIRITS	Satisfactory	FALSE		0
1000 SPIRITS	Unsatisfactor	FALSE	RED	5
1000 SPIRITS	Complete	FALSE		0
1000 SPIRITS	Satisfactory	FALSE	BLUE	5
1000 SPIRITS	Satisfactory	FALSE		0
1000 SPIRITS	Unsatisfactor	FALSE	BLUE	5
1000 SPIRITS	Unsatisfactor	FALSE	BLUE	2
1000 SPIRITS	Unsatisfactor	FALSE	RED	25
1000 SPIRITS	Unsatisfactor	FALSE	RED	25
1000 SPIRITS	Unsatisfactor	FALSE	BLUE	5
1000 SPIRITS	Unsatisfactor	FALSE	RED	5

Figure 5.30 – Input big data spreadsheet

Since the EMR cluster is launched and ready, we will submit a Spark script to the cluster to process our data, as a step (EMR Step is a unit of work that contains instructions to manipulate data for processing in the EMR cluster).

The `file emr_process.py` Spark script is shown here:

```
import argparse
from pyspark.sql import SparkSession
def calculate_red_violations(data_source, output_target):
    with SparkSession.builder.appName("Calculate Red Health
Violations").getOrCreate() as spark:
        # Load the violation CSV data
        if data_source is not None:
            restaurants_df = spark.read.option("header",
"true").csv(data_source)
        # Create an in-memory DataFrame to query
        restaurants_df.createOrReplaceTempView("restaurant_
violations")
        # Create a DataFrame of the top 10 restaurants with the
most Red violations
        top_red_violation_restaurants = spark.sql("""SELECT
name, count(*) AS total_red_violations
          FROM restaurant_violations
          WHERE violation_type = 'RED'
          GROUP BY name
          ORDER BY total_red_violations DESC LIMIT 10""")
        # Write the results to the specified target
        top_red_violation_restaurants.write.option("header",
"true").mode("overwrite").csv(output_target)
if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument(
        '--data_source', help="Data Source location.")
    parser.add_argument(
        '--output_target', help="Output target location.")
    args = parser.parse_args()

    calculate_red_violations(args.data_source, args.output_
target)
```

Upload the Spark script, `emr_process.py`, to the S3 bucket, `s3://emr20230308/`.

Upload the source spreadsheet, `s3://emr20230308/source/food_inspection_data.csv`, and create a target S3 folder, `s3://emr20230308/target`. We are now ready for the next step.

Click the cluster, click the **Steps** tab, and then select **Add step**. Fill in the step details as shown in *Figure 5.31*:

- For **Step type**, choose **Spark application**.
- For **Name**, enter `Spark application`.
- For **Deploy mode**, keep the default "**Cluster**".
- For **Application location**, enter the Spark script, `s3://emr20230308/emr_process.py`.
- In the **Arguments** field, enter the following arguments and values:
  - `--data_source s3://emr20230308/source/food_inspection_data.csv`
  - `--output_target s3://emr20230308/target`

This is shown in the following screenshot:

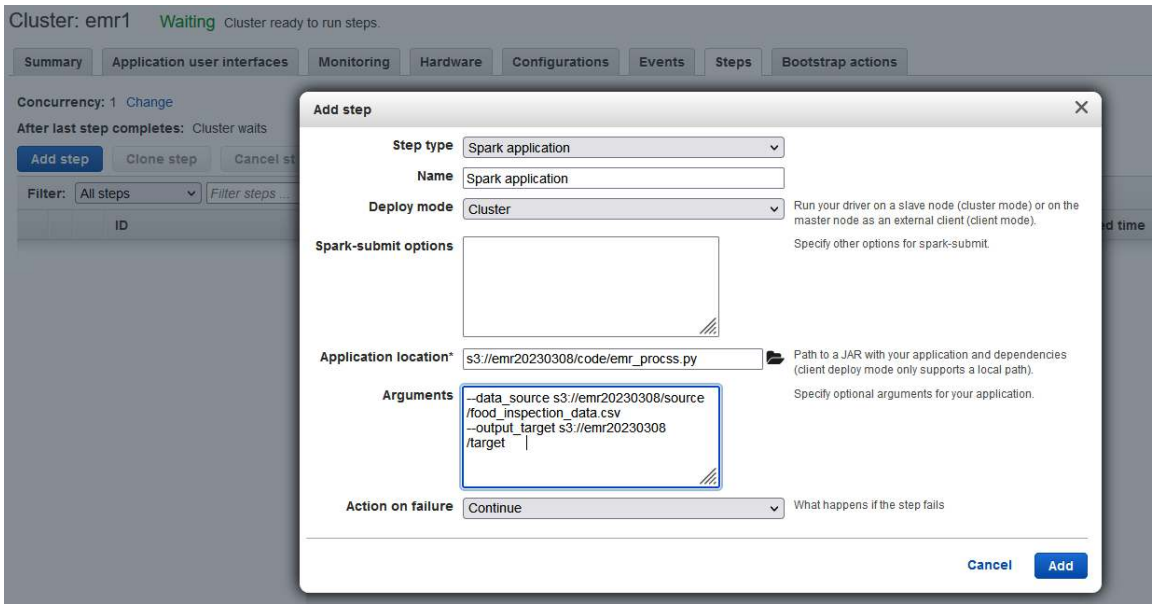


Figure 5.31 – Fill in the step details

Click **Add** to submit the Spark step. Initially, the status will show as **Pending**, but it will change to **Running** and then **Completed**, as shown in *Figure 5.32*:

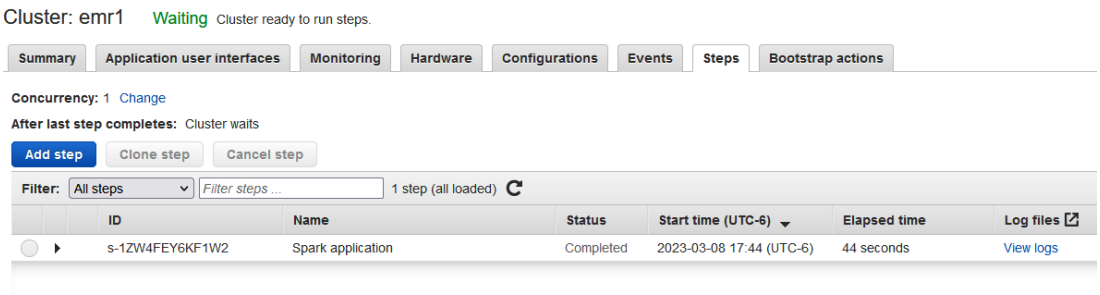


Figure 5.32 – Spark step is complete

Now, you can view its output results in your Amazon S3 output folder. As shown in *Figure 5.33*, there are two files generated from the Spark step:

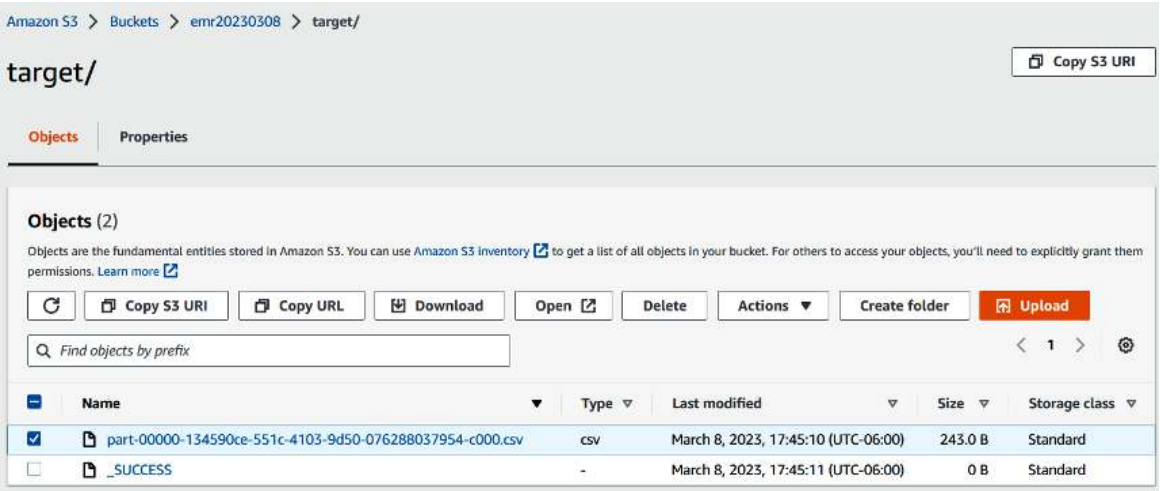


Figure 5.33 – Spark step target folder

Download the results to your local filesystem. *Figure 5.34* shows the content of the CSV spreadsheet – the EMR processing result of the submitted Spark step:

	A	B	C
1	name	total_red_violations	
2	PCC COMMUNITY MARKETS	251	
3	MCDONALD'S	177	
4	SAFEWAY INC #1508	143	
5	HIMITSU TERIYAKI	128	
6	NASAI TERIYAKI	119	
7	SAFEWAY # 1965	119	
8	SAFEWAY STORES INC #1550	118	
9	SAFEWAY STORE #1477	115	
10	SAFEWAY #1993	115	
11	FRED MEYER INC #179	111	
12			

Figure 5.34 – Spark step processing result

3. **Submit a Hive step to process CloudFront data**

With the same procedure, we submit a Hive step to process some CloudFront data, with the following code, source, target, and step submission shown in *Figure 5.35*:

`s3://us-east-1.elasticmapreduce.samples/cloudfront/code/Hive_CloudFront.q`

`s3://us-east-1.elasticmapreduce.samples`

`s3://emr20230308/hive/target`

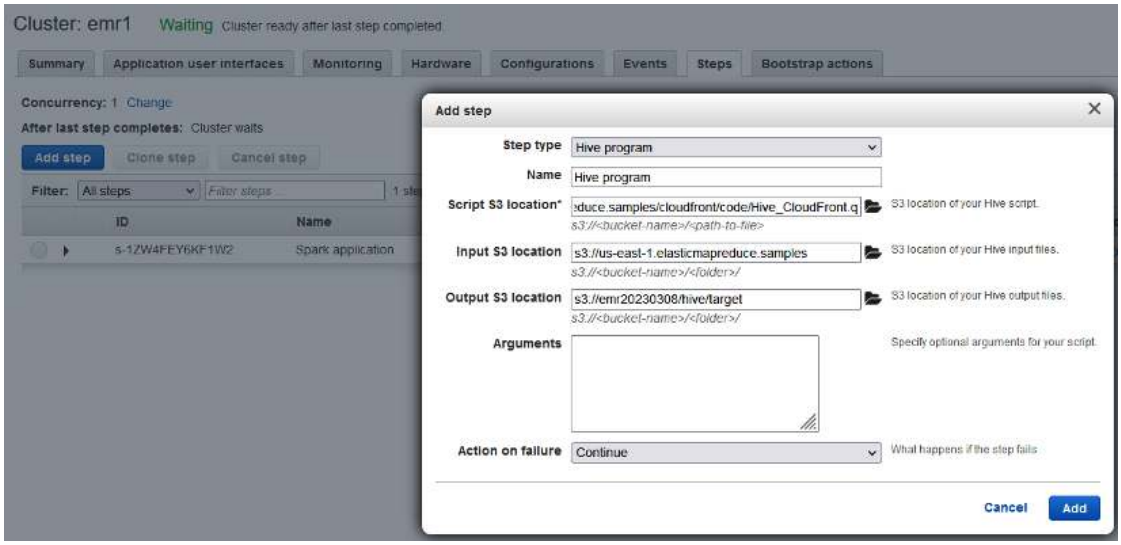


Figure 5.35 – Hive step submission

As shown in *Figure 5.36*, there are two files generated from the Hive step. Download them to your local filesystem. Please compare the source data and target processing results.

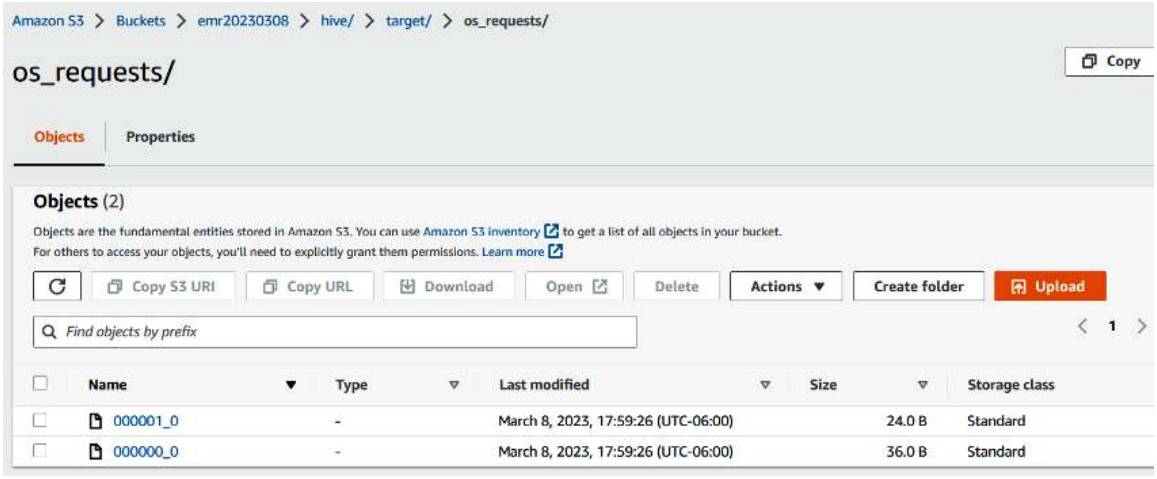


Figure 5.36 – Hive step output files

To recap, we have launched an EMR cluster and added two steps for the big data process: a Spark step and a Hive step. As you can see, both steps of big data processing takes less than a minute to process a big amount of data.

Summary

In this chapter, we explained big data analytics in the AWS cloud: ingestion, storing, processing, and visualization. We introduced AWS big data services including Glue, Kinesis, Athena, EMR, and QuickSight. We have demonstrated big data ingestion using AWS Glue and Kinesis, big data processing using Amazon Athena and EMR, and visualization using Quicksight, S3 stores the big datasets.

In the next chapter, we will discuss the Amazon machine learning services.

Practice questions

Questions 1-8 are based on the data analytics pipeline in the AWS cloud shown in *Figure 5.37*. An engineer is designing a pipeline that will ingest long-term, big-volume streaming data from the web using Kinesis Data Streams, then make two copies: one copy pass to Kinesis Firehose and stored in an Amazon S3 bucket, the other data copy will be processed with Amazon EMR and then queried by Athena and visualized using Amazon QuickSight. Performance and costs are the main factors to be taken into account.

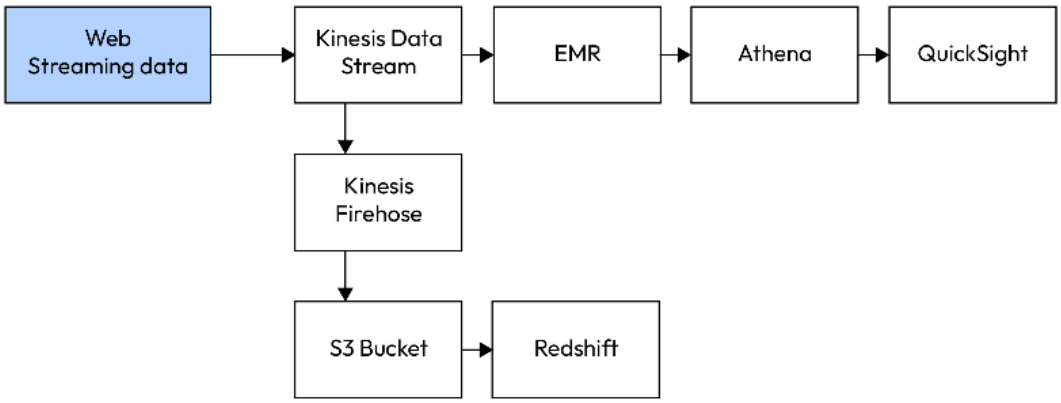


Figure 5.37 – Data analytics pipeline in the AWS cloud (redraw)

1. What instances would you recommend for the EMR cluster?

    A. Reserved Instances for the cluster

    B. Spot Instances for core and task nodes and a Reserved Instance for the master node

    C. Spot Instances for the cluster

    D. On-demand instances for the cluster

2. What filesystem would you recommend for the EMR cluster?

    A. HDFS with a consistent view

    B. EMRFS with a consistent view

    C. HDFS

    D. EBS on the cluster nodes

3. A senior manager needs to see the web data traffic trends for weekends from QuickSight. What is your recommendation?

    A. A line graph plotting traffic versus time

    B. A pie chart plotting traffic per day of the week

    C. A heatmap overlay to show the volume of traffic

    D. A bar graph plotting location versus web traffic

4. Several rows in a Redshift table were accidentally deleted. What is your recommendation to restore the rows from a snapshot?

    A. Restore from the snapshot to a new Redshift cluster. Truncate the table in the original cluster and copy the table from the new cluster.

B. Restore to a new table from the snapshot. Drop the original table and rename the new table to the original table name.

C. Restore from the snapshot to a new cluster. Create a link between the two clusters and copy the data from the new cluster.

D. Restore from the snapshot and overwrite the original table.

5. Right after the data is output from Kinesis Firehose to S3, it will be sent to Redshift, and the S3 data will be kept for 30 days before moving to the Infrequent Access tier and being deleted in a year. What's your recommendation for the S3 data storage?

A. Archive it in Glacier till deletion

B. Use standard S3 till it's deleted

C. Leverage S3 life cycle management

D. Enable S3 versioning

6. New regulation requires that only authorized staff can access customers' Social Security numbers in the data ingested from the social media.

What's your recommendation for handling this sensitive information?

A. Mask the Social Security numbers in Kinesis data stream ingestion

B. Store a cryptographic hash of the Social Security numbers

C. Encrypt the Social Security numbers with a key and share the key only with authorized staff

D. Encrypt the Social Security numbers with a key and give the decryption key only to authorized staff

7. A Spark step will be added to the EMR cluster for machine learning model training. What instances will you choose?

A. C instance types

B. R instance types

C. T instance types

D. D instance types

8. Senior management needs QuickSight visual reports every Monday morning, regarding the previous weekend's social media engagement. What's your recommendation for the EMR cluster?

A. Keep the EMR up and generate reports every Monday morning

B. Use a transient EMR cluster to generate reports every Monday morning

C. Use Spark Streaming on EMR to aggregate and generate reports every Monday morning

D. Use the EMR cluster to aggregate media data each night and use QuickSight to report on Monday morning

Answers to the practice questions

1. B
2. B
3. A
4. B
5. C
6. B
7. A
8. B

Further reading

For further insights into what you've learned in this chapter, refer to the following links:

- `https://aws.amazon.com/glue`
- `https://aws.amazon.com/kinesis/`
- `https://aws.amazon.com/athena/`
- `https://aws.amazon.com/emr/`
- `https://aws.amazon.com/quicksight/`
- `https://docs.aws.amazon.com/emr/index.html`
- `https://docs.aws.amazon.com/quicksight/index.html`